

Network Segmentation And Access Control With Firewalls

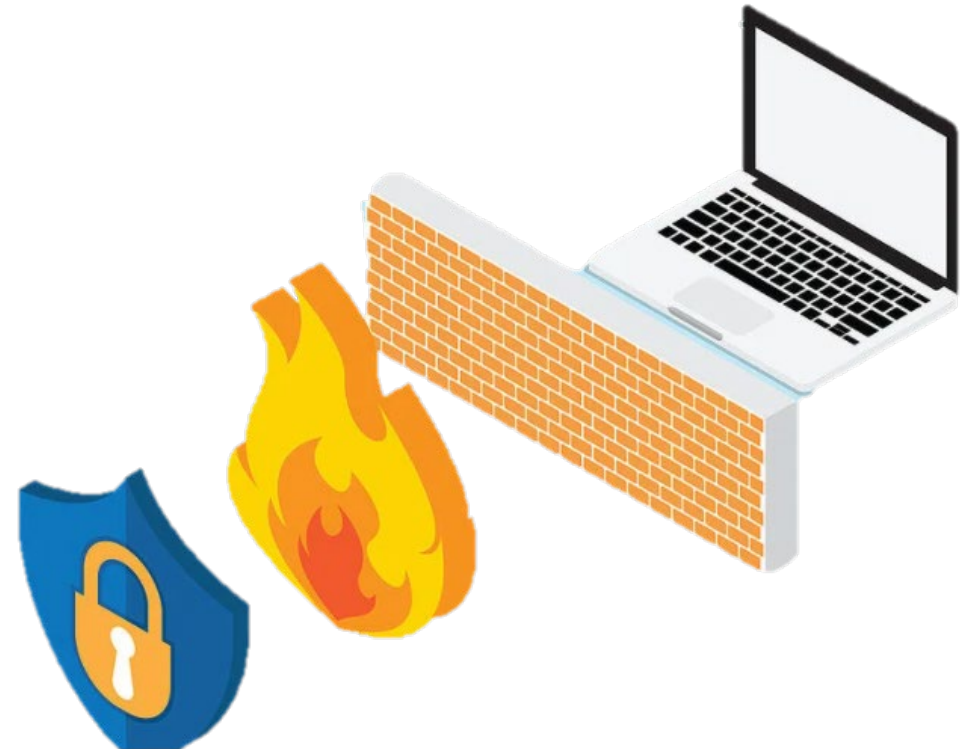


ICT2203 Networks Security
LAB 6 NOTES

LEARNING OUTCOMES

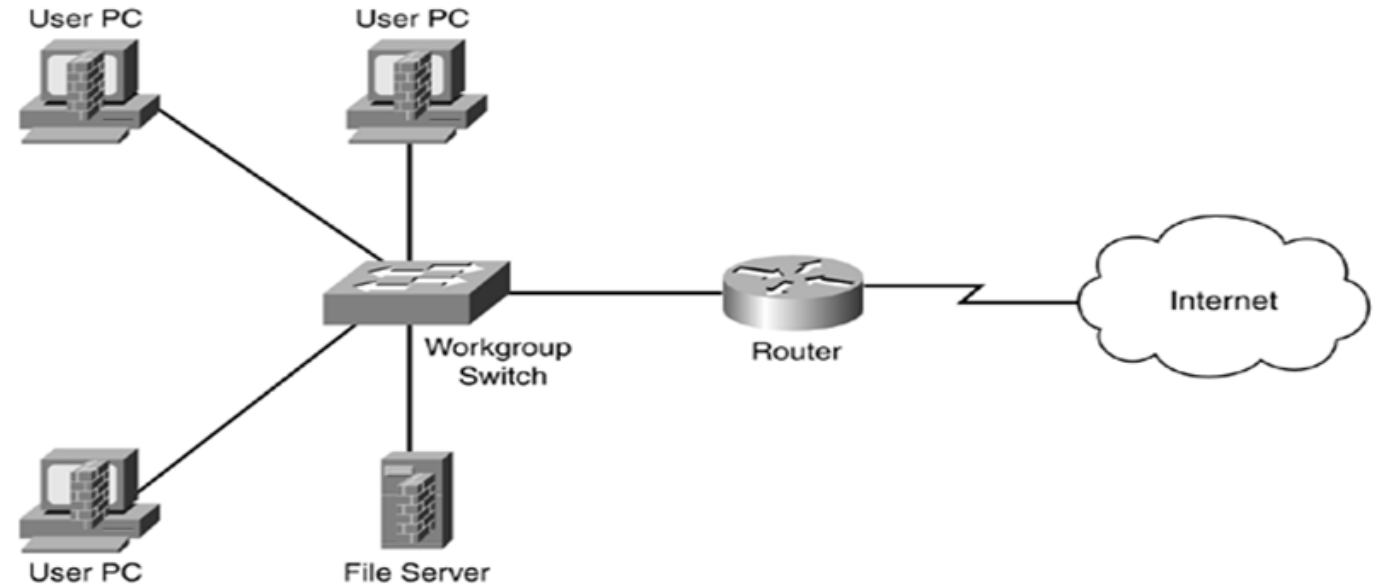
In this lab you will learn about

- Firewall technologies used
- Segmenting networks into different security zones
- Configuring a stateful firewall using Cisco ASA
- Enforce access control policy



WHAT IS A FIREWALL?

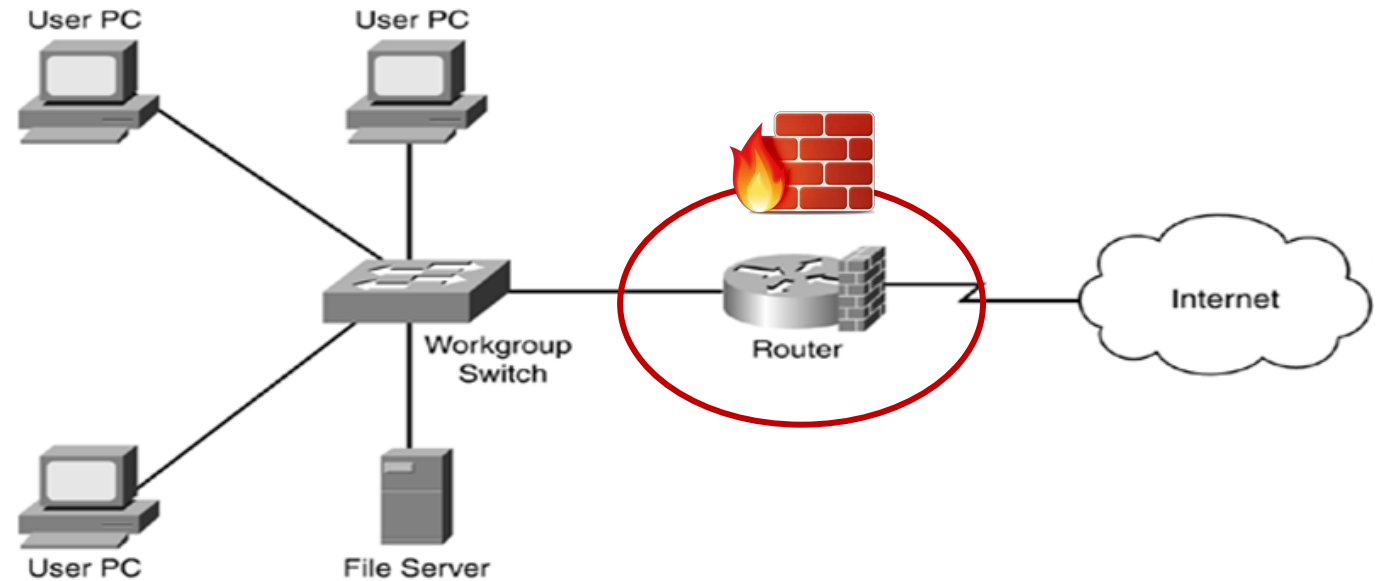
You are likely to be familiar with **personal firewall** or **host-based firewall** that is implemented as software running on the PC; e.g. Windows firewall.



However, it can be difficult to manage if there are hundreds or thousands of hosts, each running its own personal firewall!

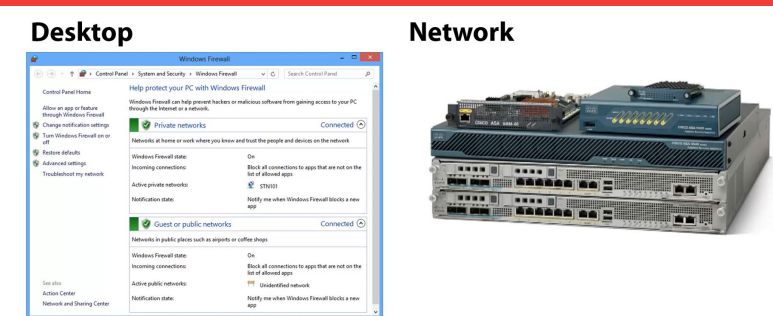
WHAT IS A FIREWALL?

Essentially, a **firewall** is a device or software that segments networks into different **security zones** and enforces **access control policy** on traffic crossing between them.



To protect a network, it is more manageable to have **network firewall** which is usually implemented as a

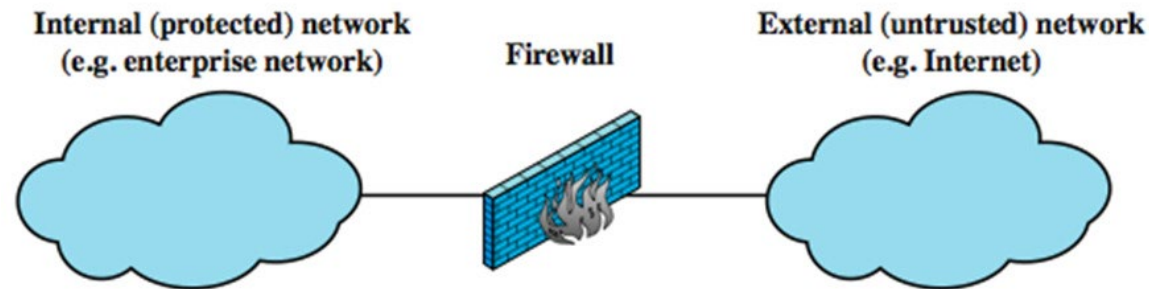
- Dedicated hardware device
 - Cloud firewall (Firewall-as-a-service - FaaS)
- placed at the **network perimeter**



MORE ON FIREWALLS...

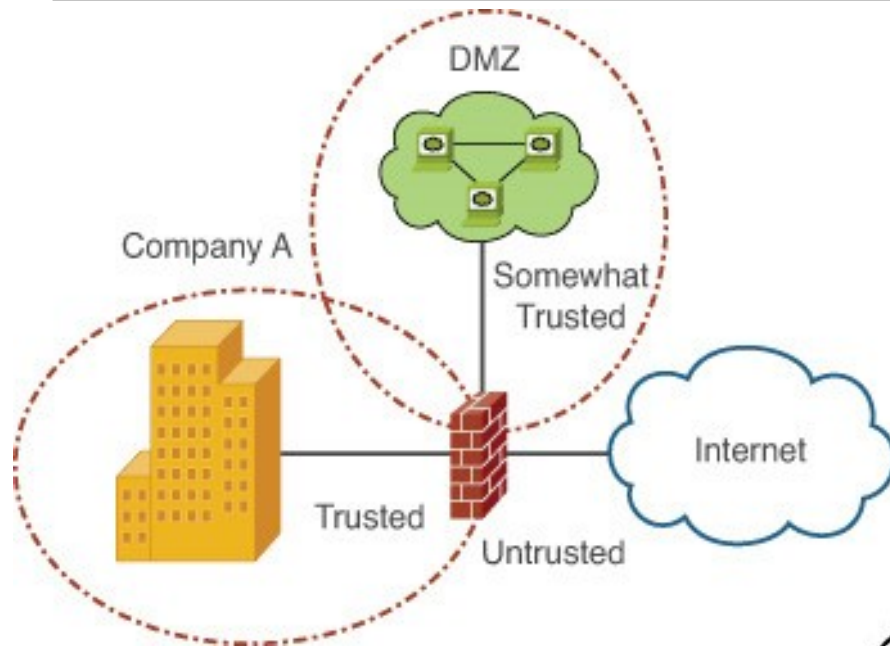
To be effective in enforcing **access control** policy, a **firewall** must be:

- **Only transit point** between different security zones
- **Resistant to attacks** itself

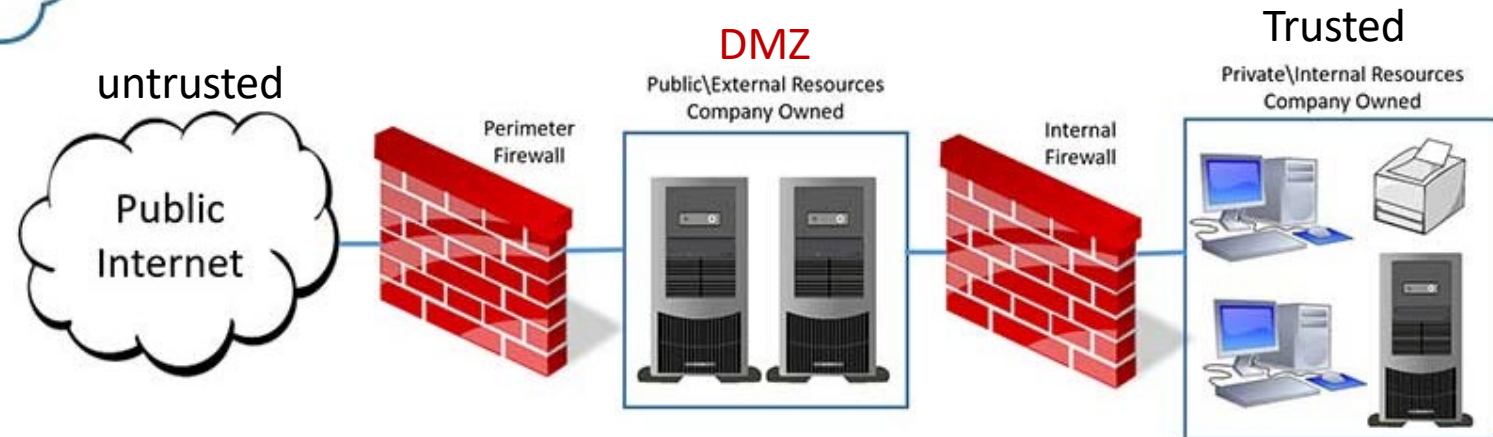


Limitation: Firewall cannot enforce access control policy on network traffic within the same security zone that do not pass through the firewall.

To segment networks into **more than two security zones**, an enterprise may implement **one** or **more firewalls** as follows:



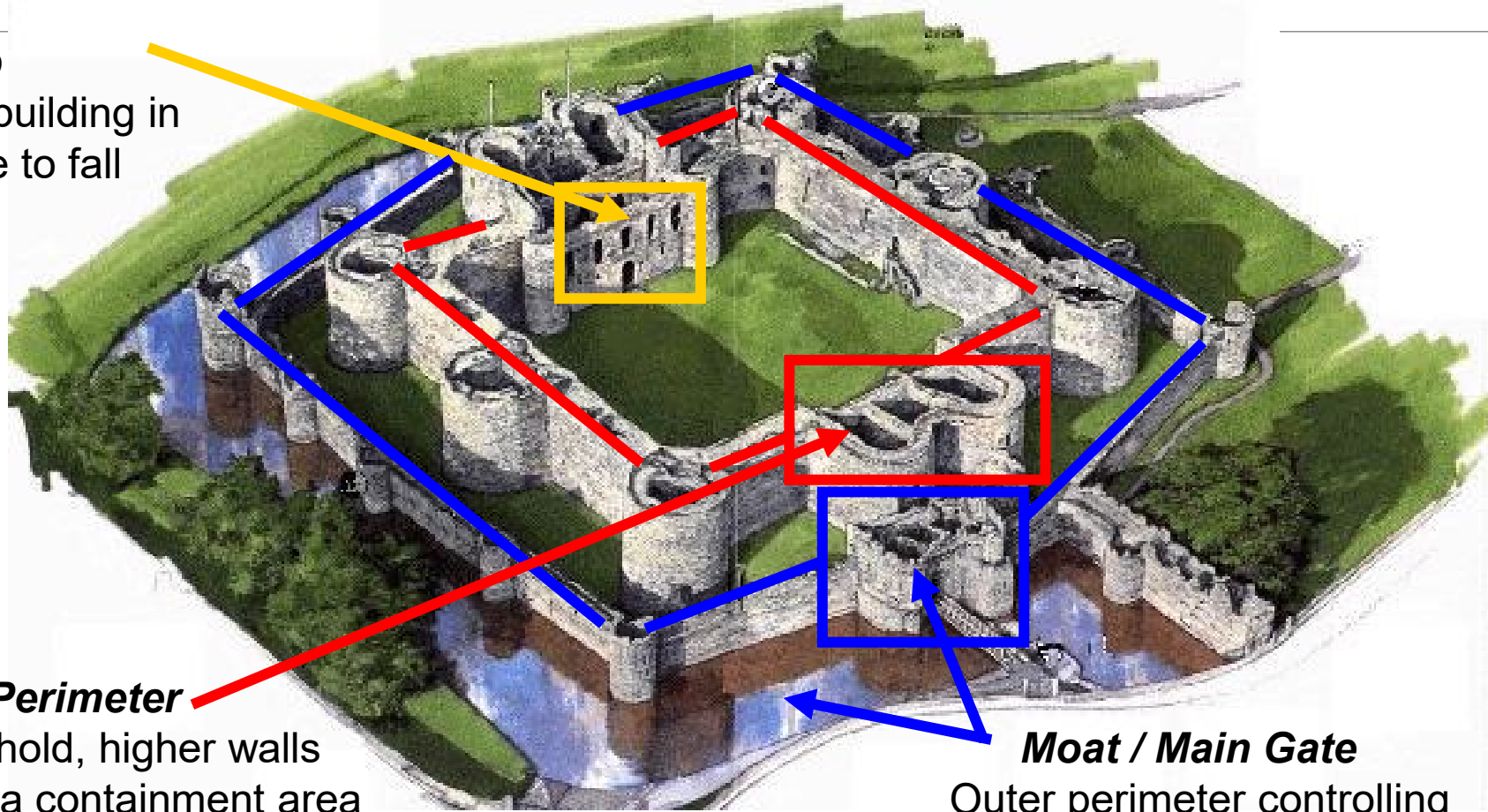
Demilitarized zone (DMZ) allows an organization to allow (untrusted) external hosts limited access to organization's resources, e.g. website.



CASTLE DEFENSES: ANALOGY

Keep

Last building in castle to fall



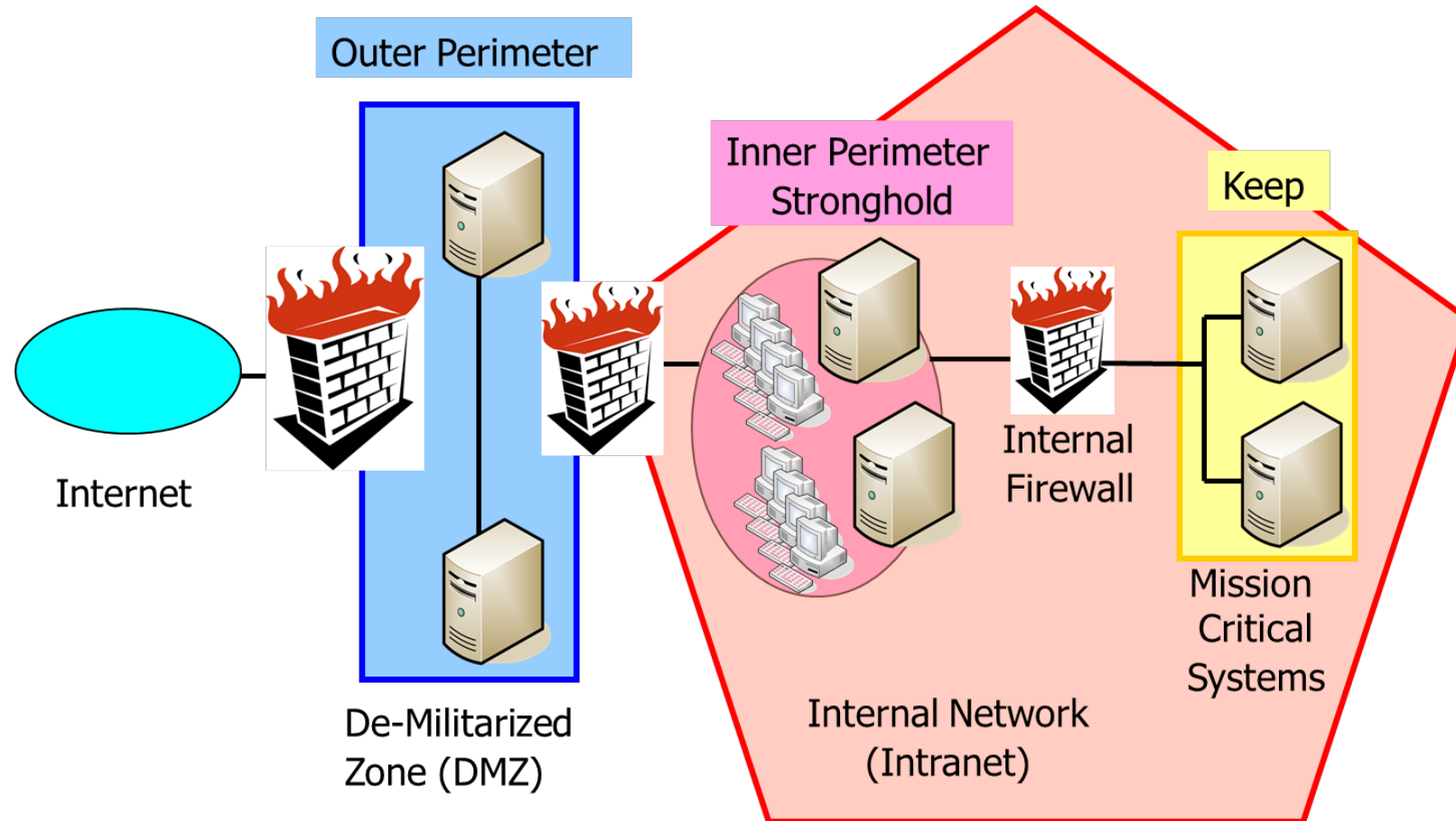
Inner Perimeter

Stronghold, higher walls create a containment area between Inner & Outer Perimeters

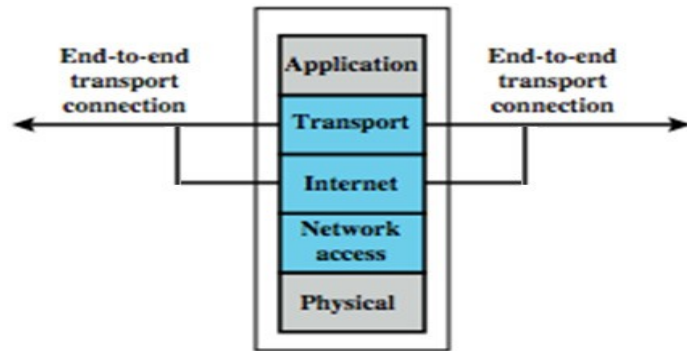
Moat / Main Gate

Outer perimeter controlling castle access

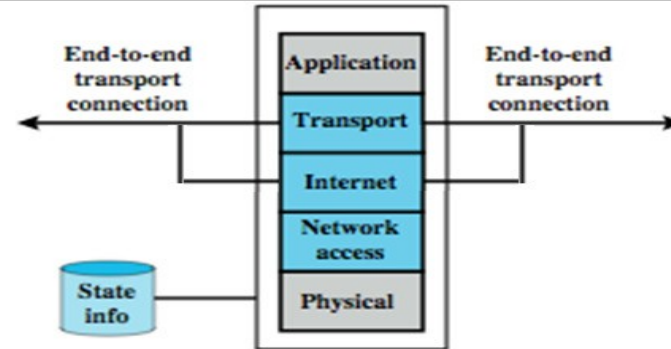
NETWORK DEFENCES: A CASCADE OF SECURITY ZONES



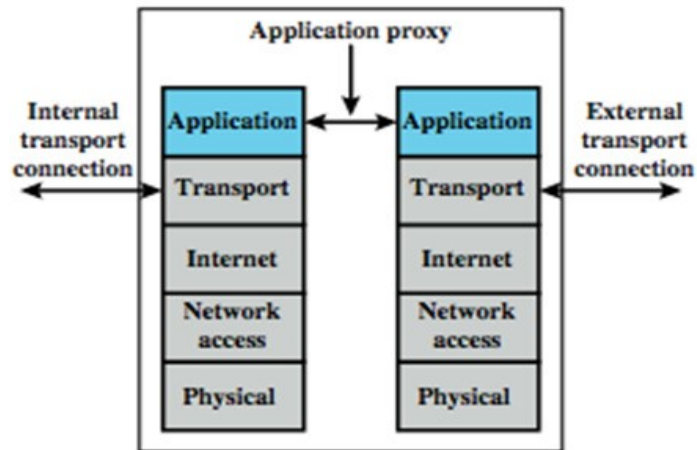
Broadly, **firewall** technologies may be divided into 4 different categories



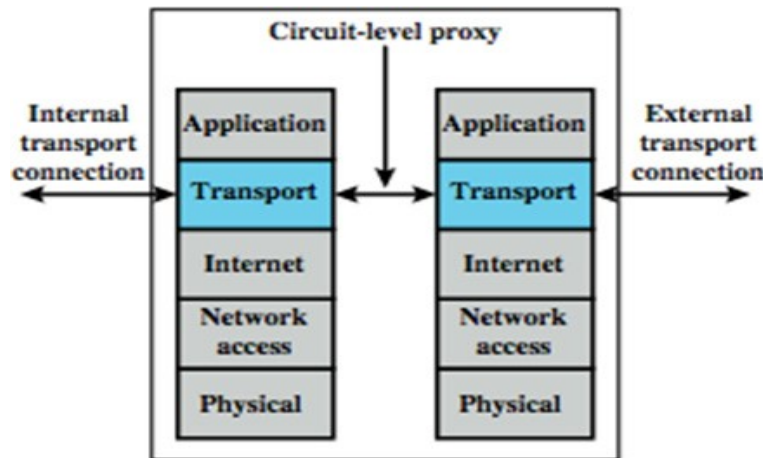
(A) Packet filtering firewall



(B) Stateful inspection firewall



(C) Application proxy firewall



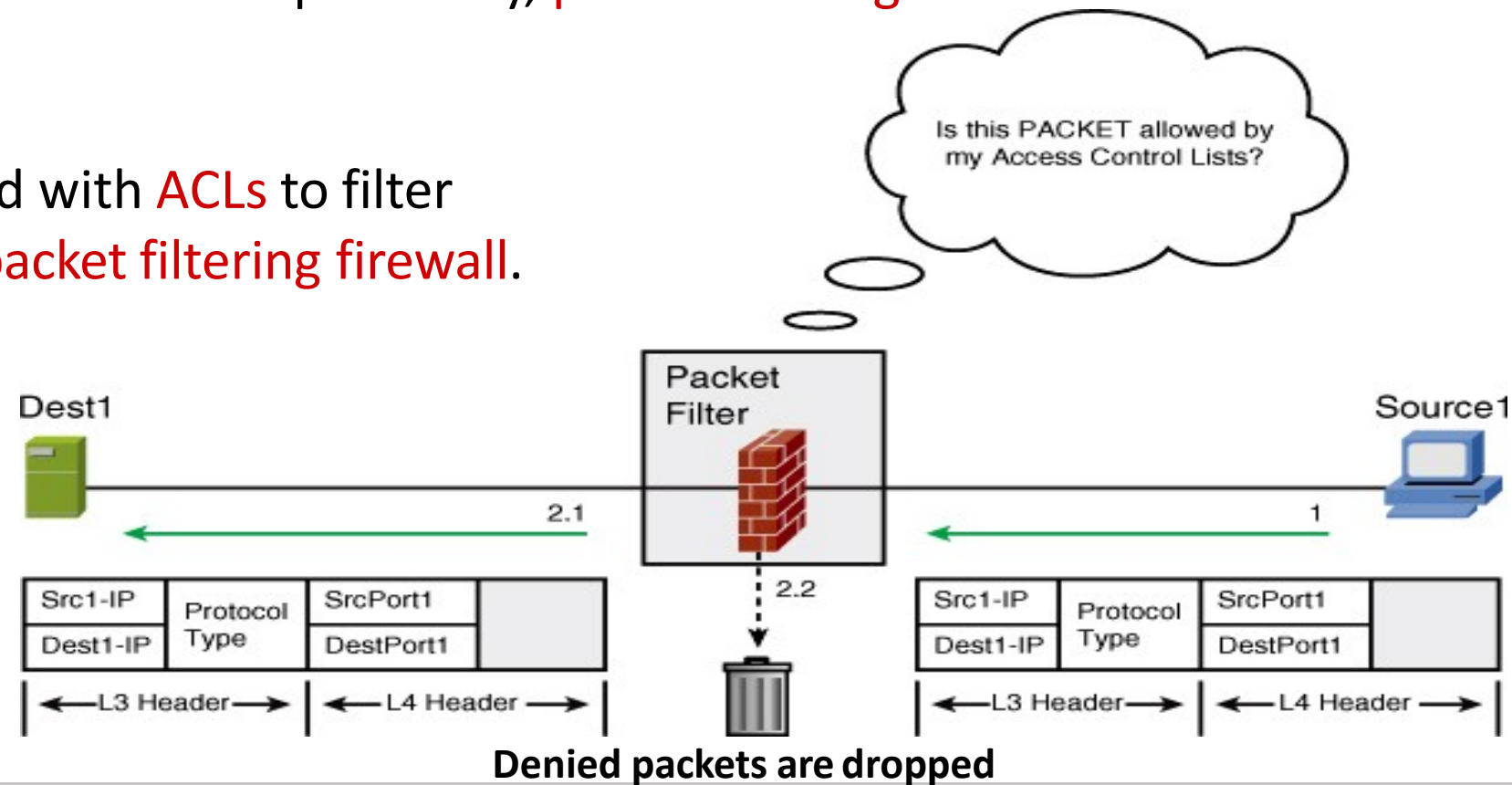
(D) Circuit-level proxy firewall

(A) **PACKET FILTERING FIREWALL**: A packet filtering firewall applies a set of rules (access control) to each incoming and outgoing IP packet and then forwards or discards the packet.

Since each IP packet is examined independently, **packet filtering firewall** is also called **stateless firewall**.

Note: A **router** configured with **ACLs** to filter packets is technically a **packet filtering firewall**.

Access control policy rules are implemented as **ACLs** to filter **each IP packet** independently



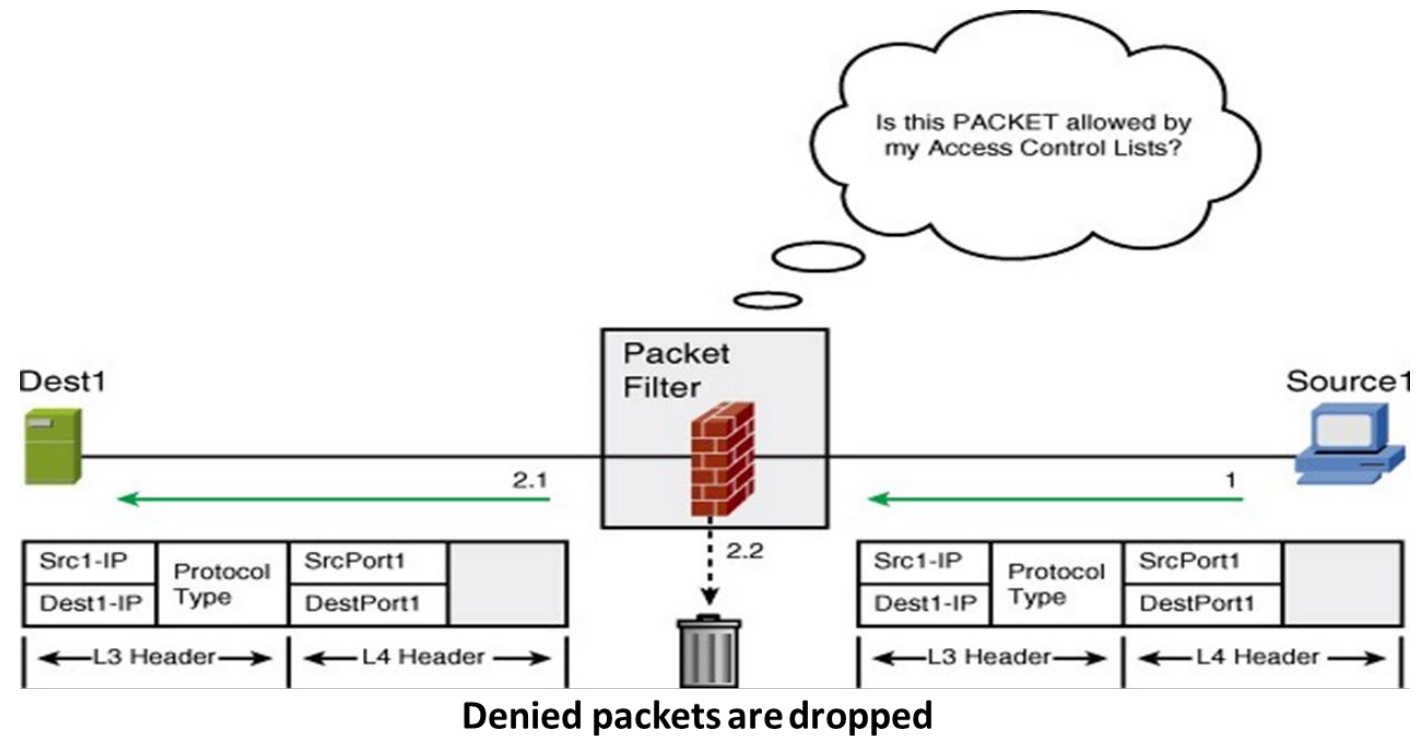
(A) PACKET FILTERING FIREWALL

✓ Advantage :

- Cost
- Low resource usage
- Best suited for smaller network

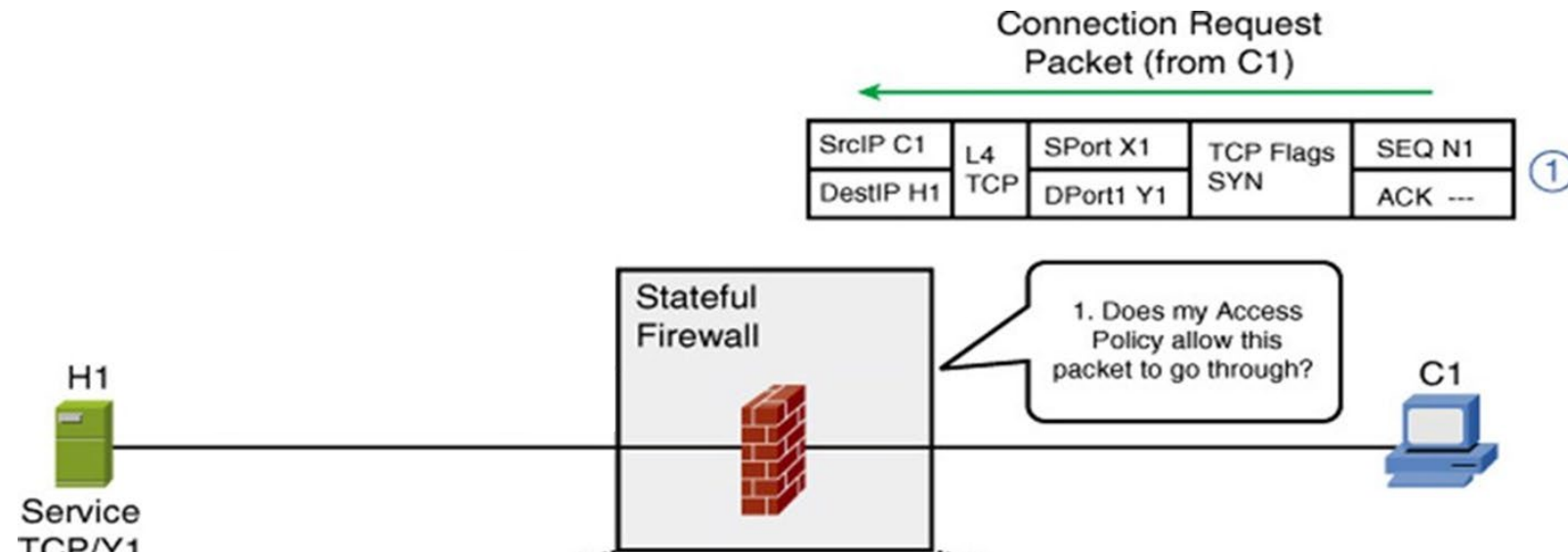
✗ Disadvantage :

- Can work only on the packet header
- Do not support complex rule-based support
- Vulnerable to spoofing

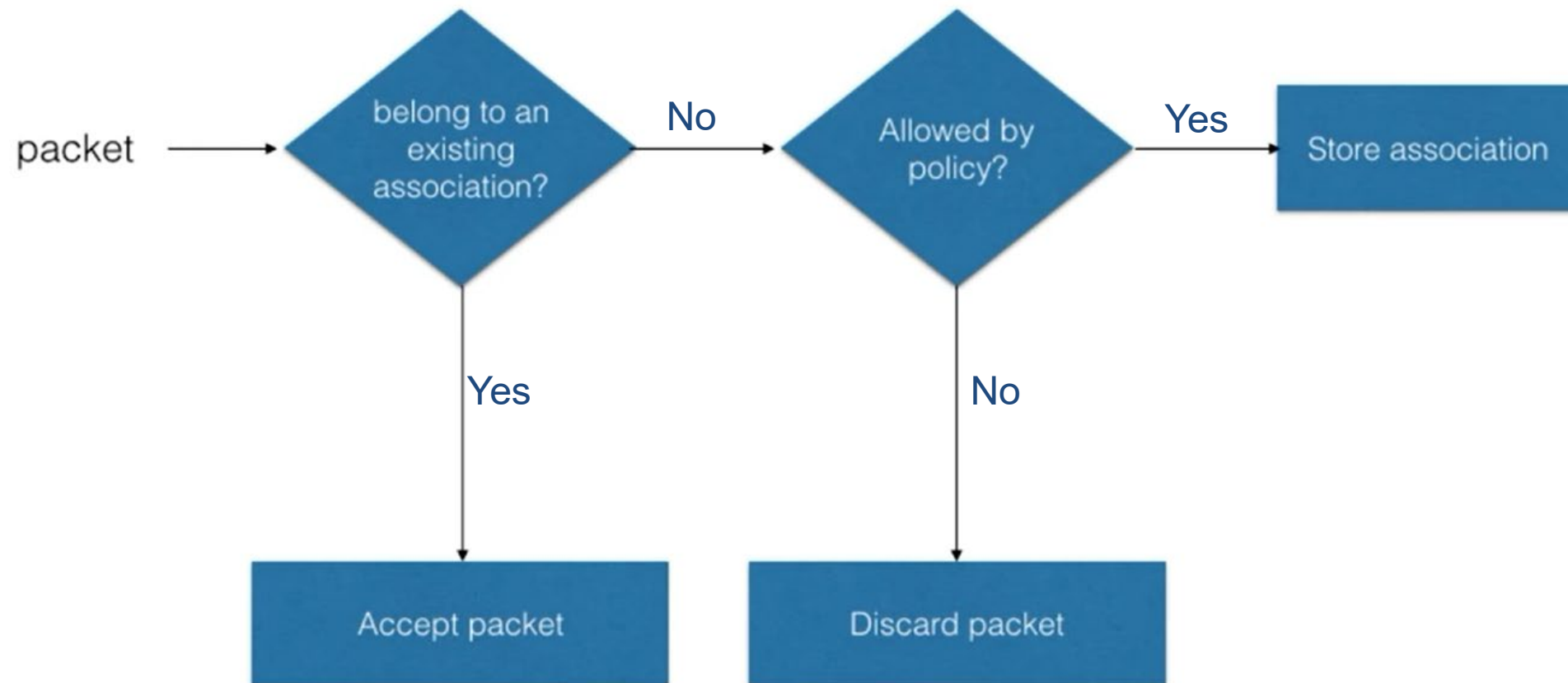


(B) **STATEFUL INSPECTION FIREWALLS:** To provide better security, stateful inspection firewalls are developed to maintain connection information in **state-table** for **packet filtering** and are commonly in use today.

- Tightens up the rules for TCP traffic by creating a directory of all outbound TCP connections.
- Reviews the same packet information as a packet filtering firewall, but also records information about TCP connections in the state table.



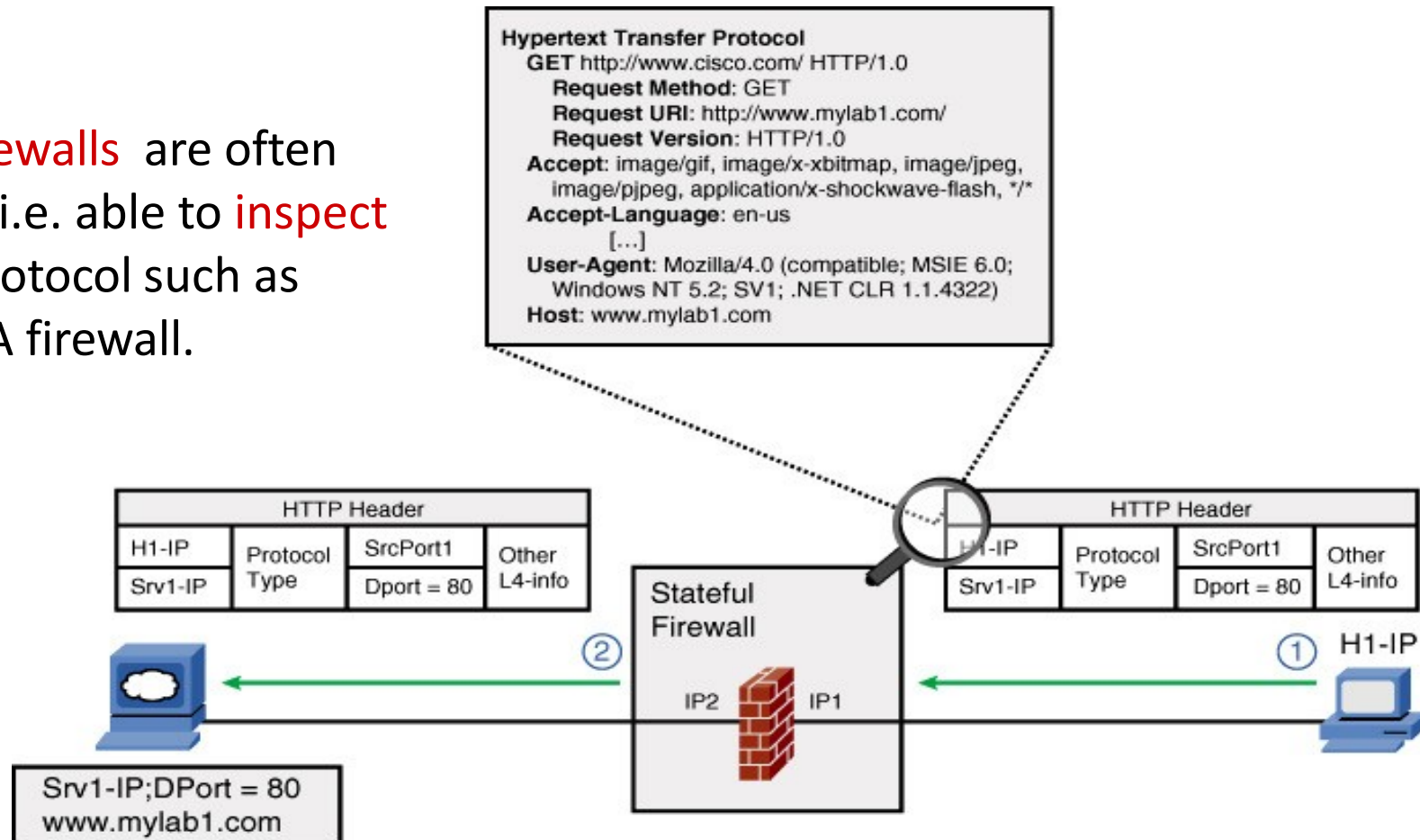
(B) STATEFUL INSPECTION FIREWALLS



(B) STATEFUL INSPECTION FIREWALLS

✓ Advantage:

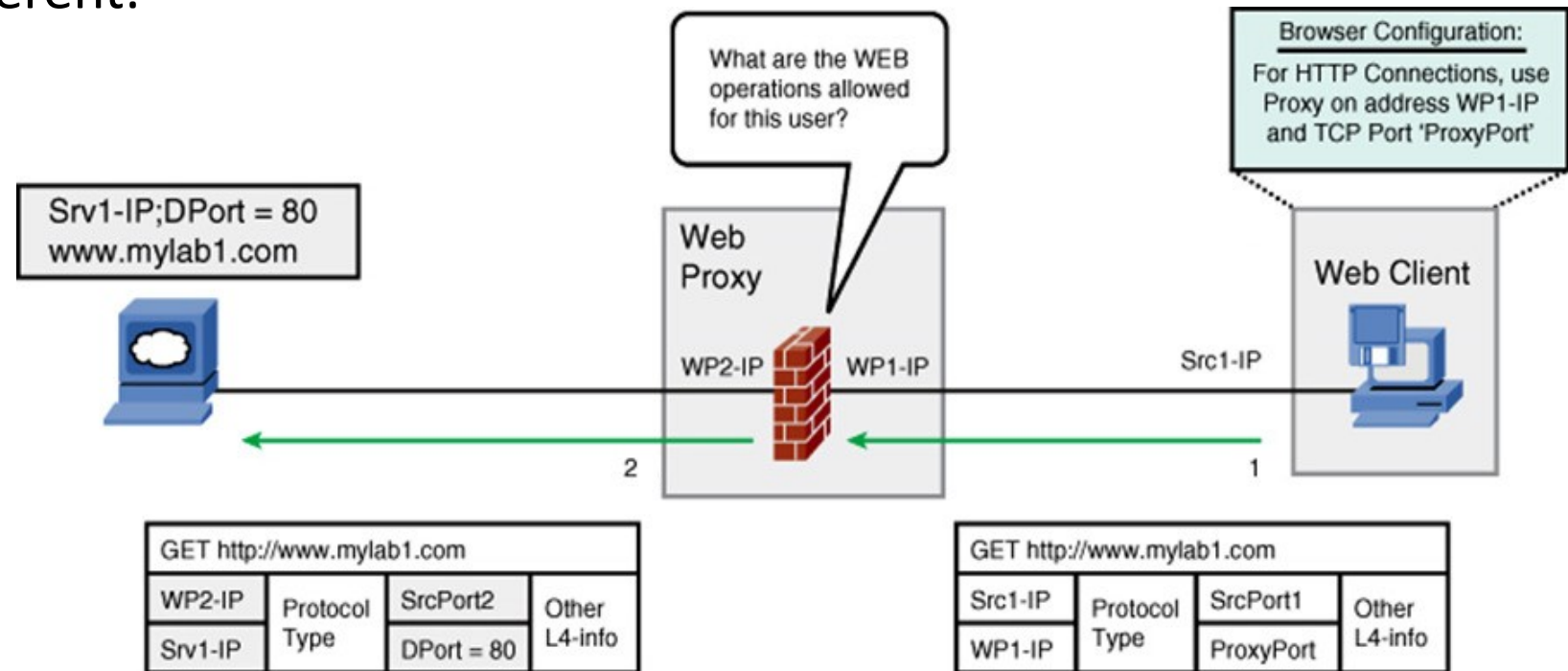
Modern **stateful firewalls** are often **application-aware**, i.e. able to **inspect application-level** protocol such as HTTP, e.g. Cisco ASA firewall.



(C) APPLICATION PROXY FIREWALL, aka application layer gateway (ALG) is specially designed to filter certain application, e.g. web proxy or Web Application Firewall (WAF).

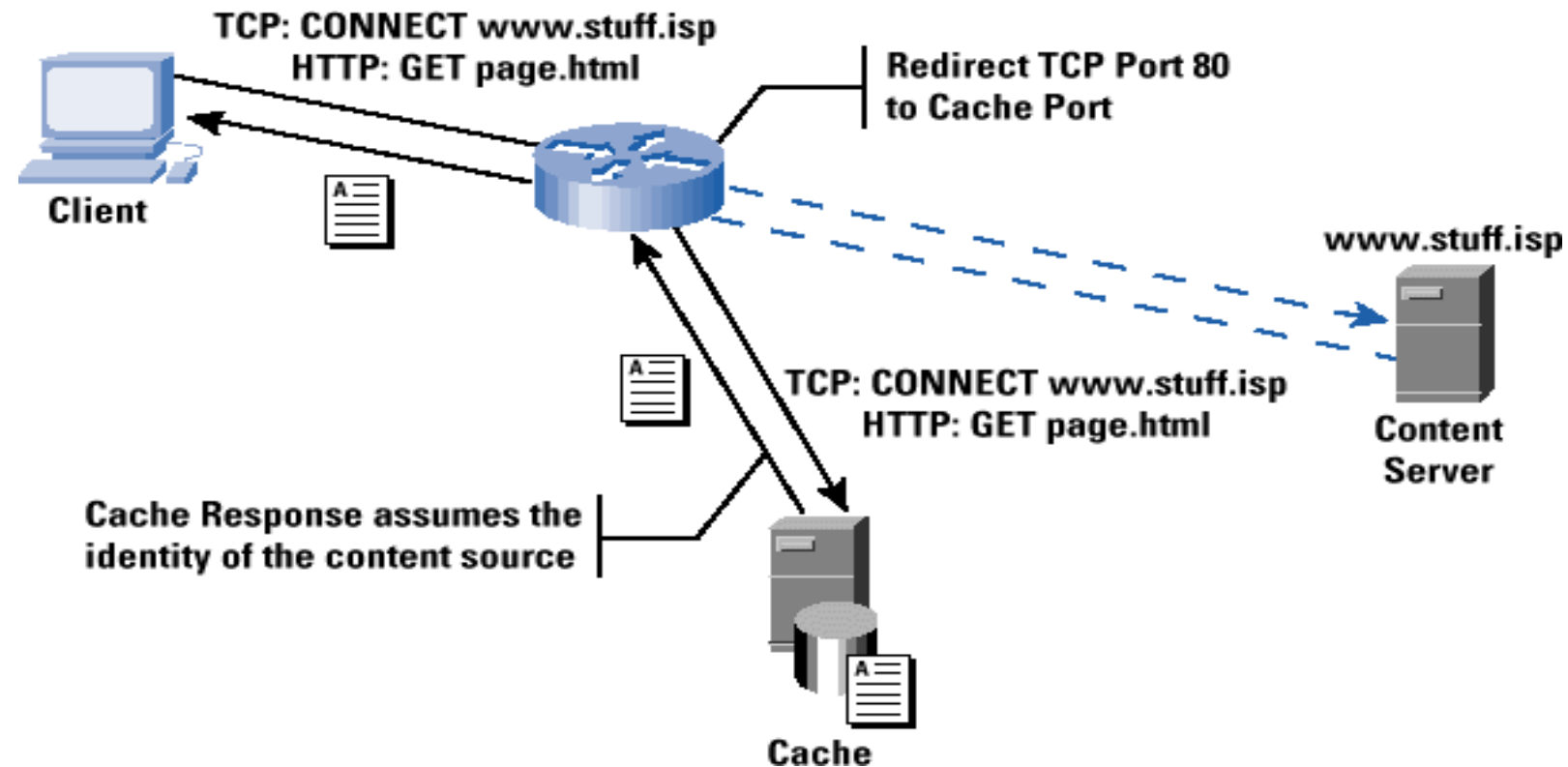
Proxy firewall and application-aware stateful firewall are different:

- Proxy firewall: two spliced connections between user and proxy, and proxy and destination server
- Stateful firewall: one direct connection between user and server



(C) APPLICATION PROXY FIREWALL may be deployed in (forward) proxy mode; e.g. transparent web proxies to restrict websites that users are allowed to access.

An example of transparent web proxy is the free open source Squid with WCCPv2 router interception which you may try in your team project. <https://wiki.squid-cache.org/Features/Wccp2>

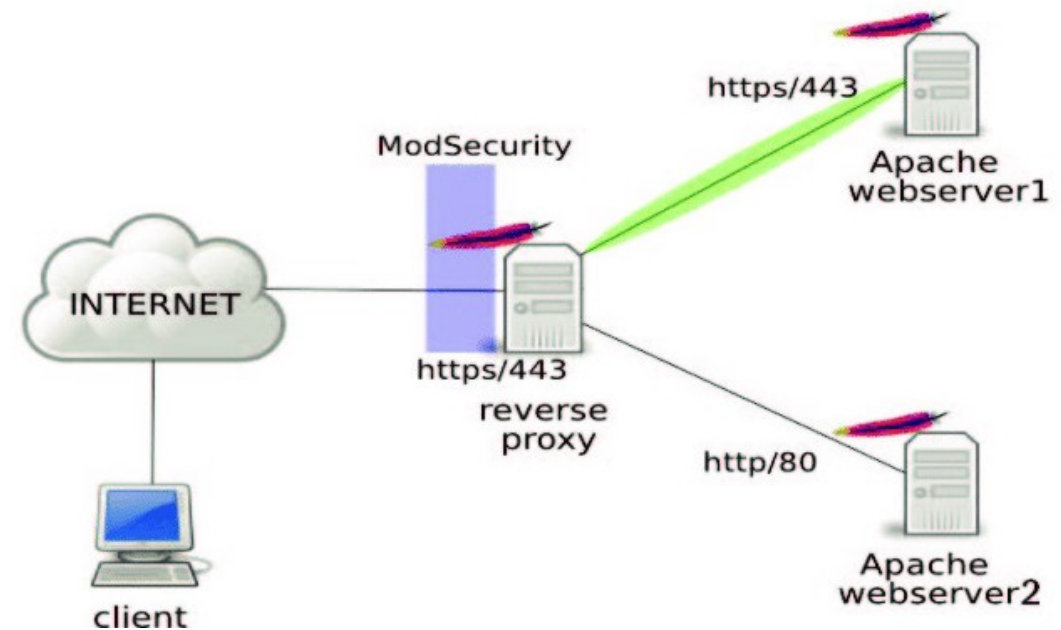


(C) APPLICATION PROXY FIREWALL may also be deployed in **reverse proxy** mode; e.g. **WAFs** to protect web servers.

- In **reverse proxy** mode, client will connect to **WAF** which acts as the web server.
- If request is non-malicious, WAF will relay request to web server and reply-back to the client

An example of **WAF** is the free open source **ModSecurity** which you may also try in your team project. <https://www.modsecurity.org/>

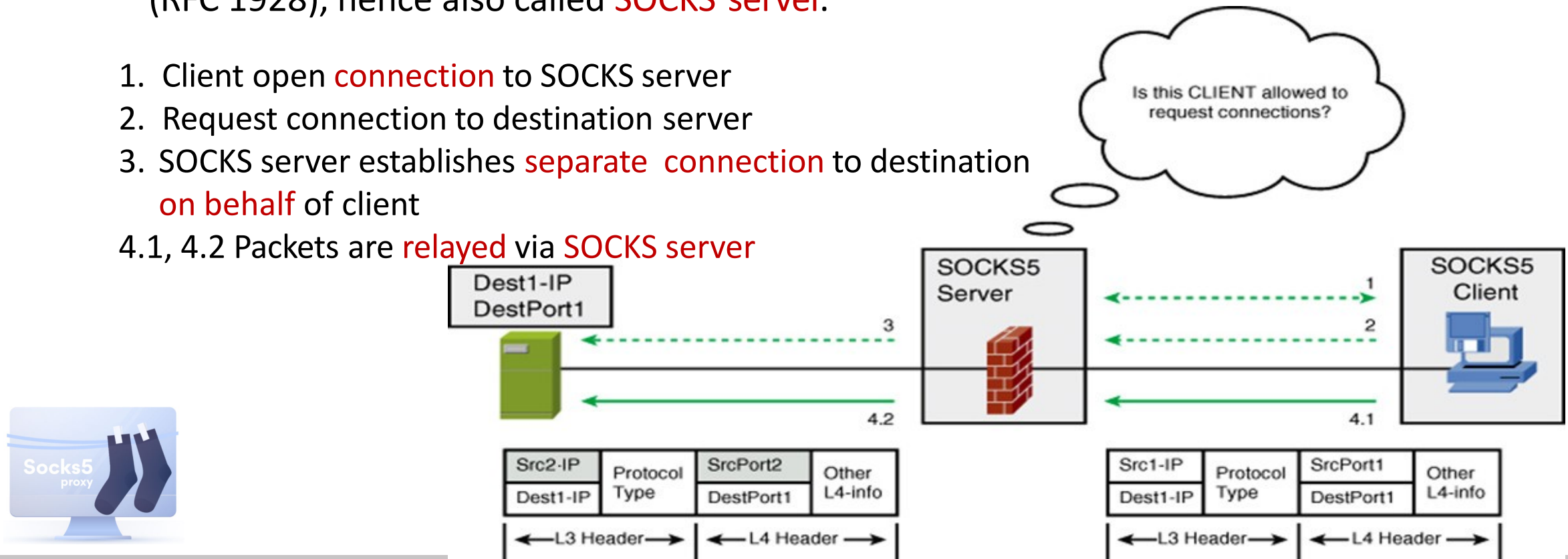
Note: **ModSecurity** may also be deployed in **embedded** mode which is on the web server itself.



(D) **CIRCUIT-LEVEL PROXY FIREWALL**, also referred as **generic proxy** works similarly by connecting to server on behalf of user but does not understand application protocols.

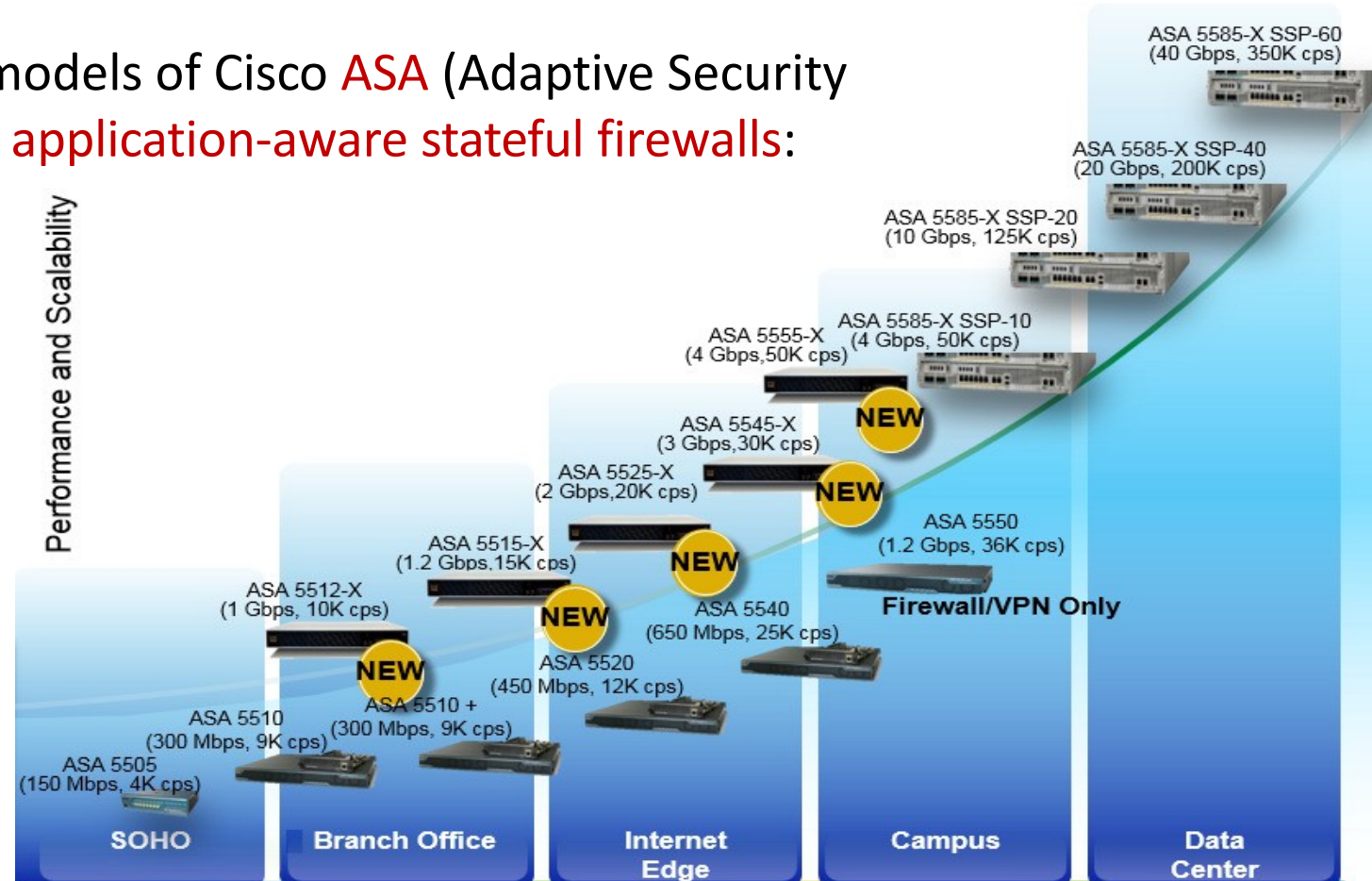
Circuit-level proxy is commonly implemented using **SOCKS5** (Socket Secure) protocol (RFC 1928), hence also called **SOCKS server**.

1. Client open **connection** to SOCKS server
2. Request connection to destination server
3. SOCKS server establishes **separate connection** to destination **on behalf** of client
- 4.1, 4.2 Packets are **relayed** via **SOCKS server**



To have a better understanding of firewall, we'll now study the Cisco ASA firewall which is an **application-aware stateful firewall**.

Common models of Cisco **ASA** (Adaptive Security Appliance) **application-aware stateful firewalls**:

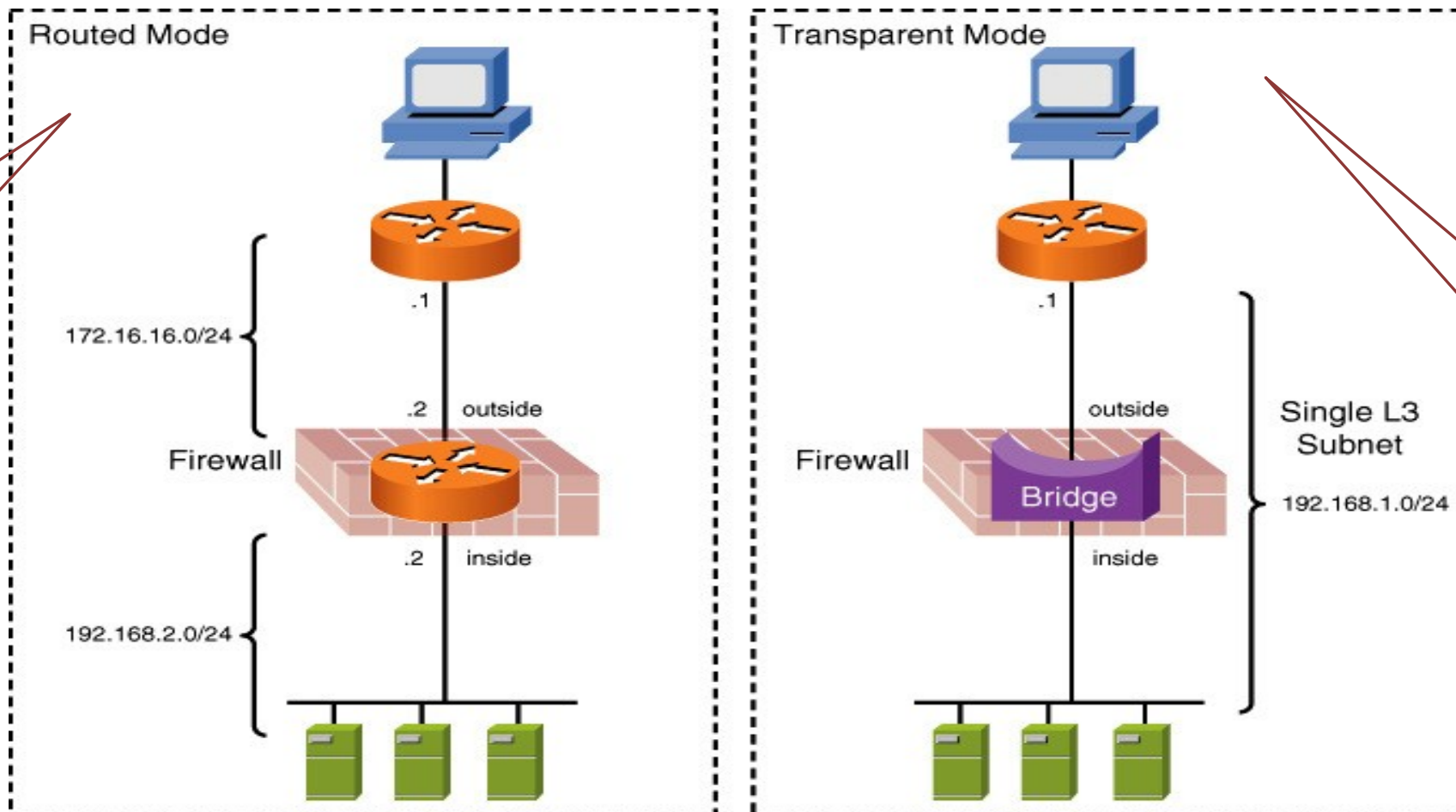


LAB 6.0



Cisco **ASA** is an example of **stateful firewall**. Using CLI commands similar as routers, configuration of **access control policy** in ASA can be done via:

- A. **Interface security-level** (mandatory)
- B. **Access Control List** (to override interface security-level if needed)
- C. **MPF** (Modular Policy Framework)
- D. (Optionally) ASA may be configured using graphical interface called Adaptive Security Device Manager (ASDM)



Routed mode:
firewall works
as a layer 3
device

**Transparent
mode:** firewall
works as a
layer 2 device

There are two ways to deploy **firewalls** into the network topology – using layer 3 (**routed mode**) or layer 2 (**transparent or bridge mode**).

A comparison of **routed** and **transparent** firewall modes in **ASAs**:

Routed mode is the **default** mode in ASA and is commonly deployed for internet edge (our focus in this module).

Routed Firewall Mode

Use when only IP packets are to be inspected.

Network readdressing is necessary across the ASA.

All interfaces can be used.

All ASA features are available.

Transparent Firewall Mode

Use when non-IP packets must be forwarded.

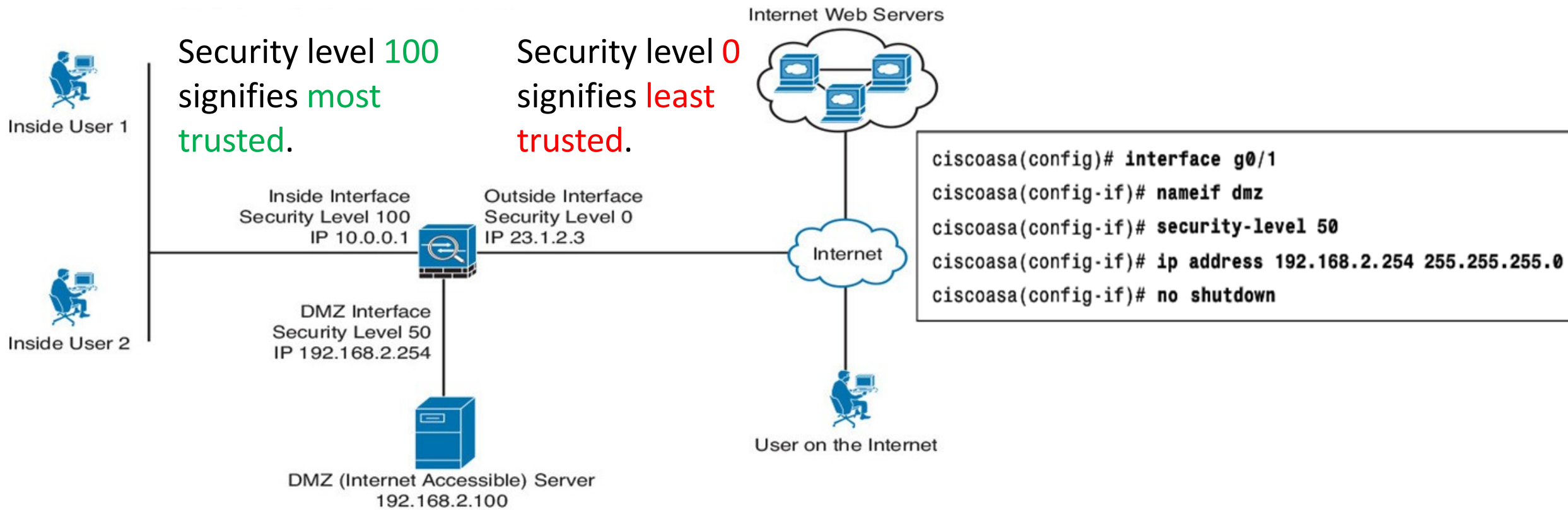
Network readdressing is not necessary.

Only 2–4 interfaces can be used per bridge group.

The following features are not available:

- Dynamic routing protocols
- Dynamic DNS
- DHCP Relay
- Multicast IP routing
- Quality of service
- VPN termination for transit traffic

LAB 6 Part (2A) In **routed mode**, ASAs are like routers and its interfaces must be configured with **IP addresses**, along with **names** and **security levels** between 0 to 100 inclusive.



To verify that the interfaces of **ASA** are up and operational after configuration, use the similar CLI command as routers but note the **slight difference** as shown:

```
ciscoasa# show interface ip brief
```

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0	23.1.2.3	YES	manual	up	up
GigabitEthernet0/1	unassigned	YES	unset	administratively down	down
GigabitEthernet0/2	unassigned	YES	unset	administratively down	down
.					
.					
Management0/0	192.168.1.1	YES	manual	up	up

```
ciscoasa#
```

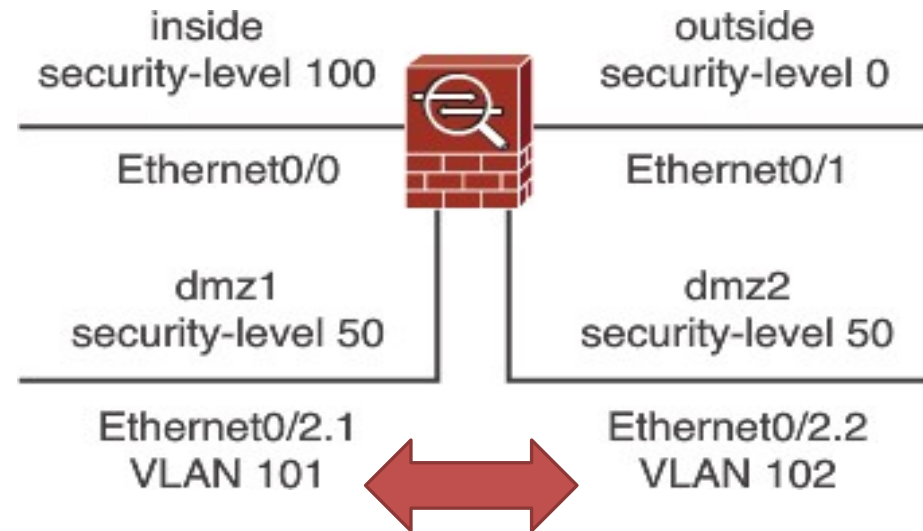
To show the details of each interface:

```
ciscoasa# show interface if_number
```

```
ciscoasa# show interface name
```

Note that if you choose to configure two or more interfaces with the **same security level**, traffic are by default **not allowed** to pass between them.

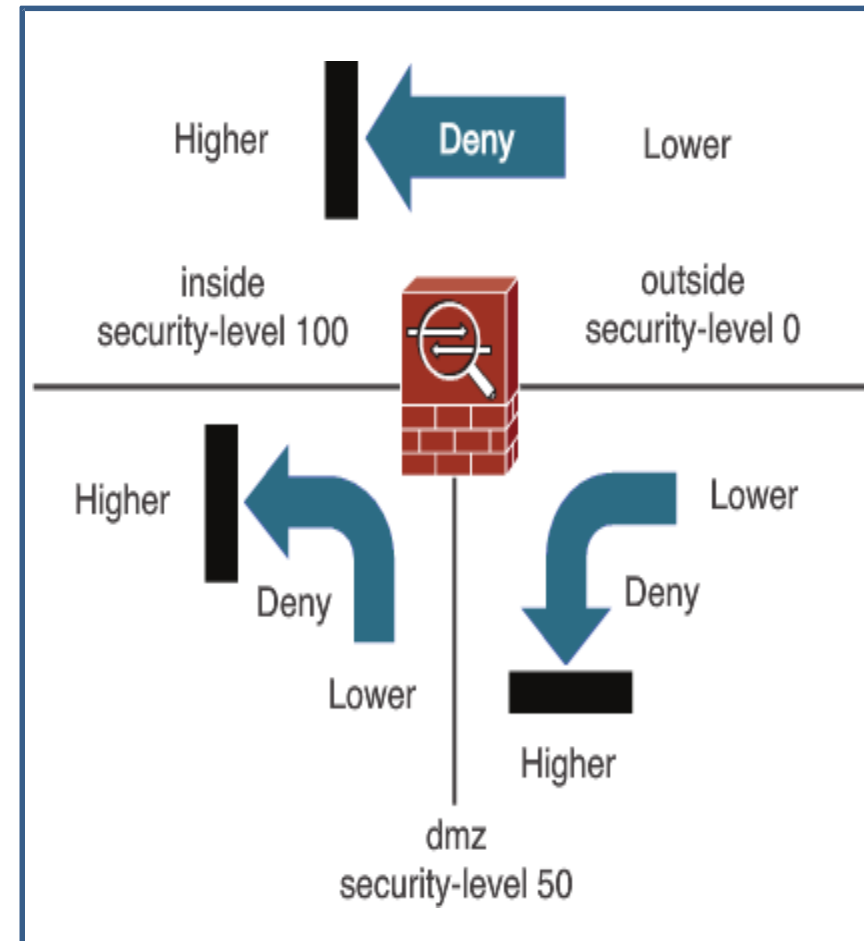
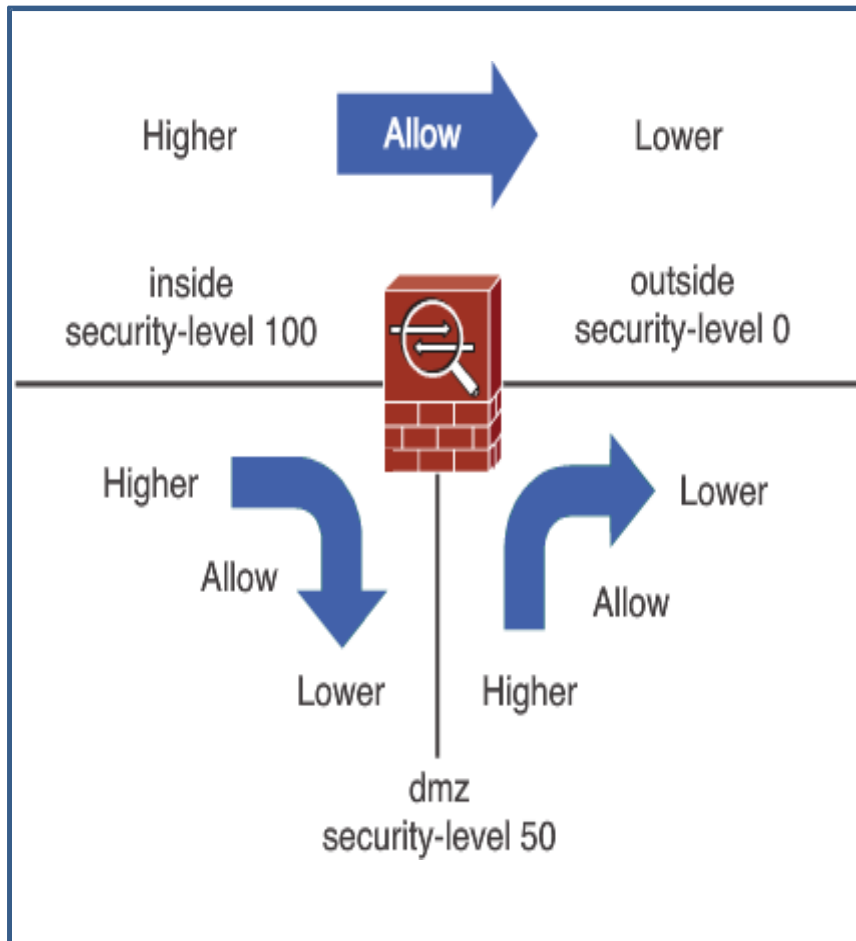
e.g.: you can use any security level between 0 and 100 to configure DMZ, but say you choose to use the **same security level 50** for **dmz1** and **dmz2** as follows:



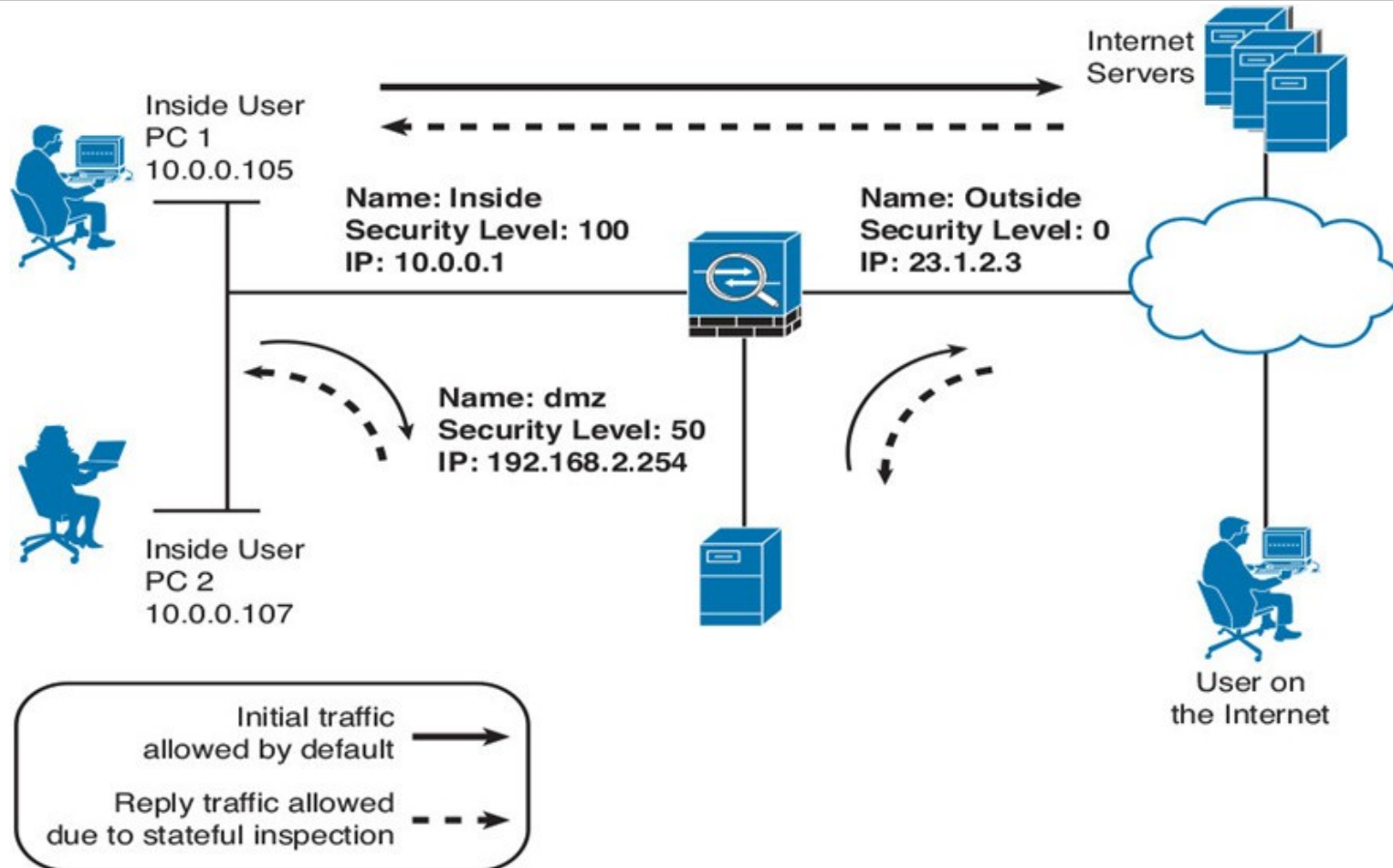
To override default security policy and allow traffic to be initiated from same security level:

```
ciscoasa(config)# same-security-traffic permit inter-interface
```

By default, traffic **initiated** from **higher security level** to **lower security level** are **allowed** but **not the reverse**.



Nevertheless, due to **stateful firewall** feature which maintains **state tables** of initiated connections, **return traffic** from **lower** to **higher security** level are automatically **allowed**.



Specifically, one of the **state tables** maintained by ASA is the **connection table** which automatically tracks the connection parameters of TCP and UDP

Protocol	Extent of Stateful Tracking
TCP	Source and destination addresses and ports, TCP flags, sequence numbers (in both directions), and idle time
UDP	Source and destination addresses and ports, idle time, and for some applications, request identifiers

To view the content of **connection table**:

```
ciscoasa# show conn
352 in use, 5985 most used
UDP DMZ 172.16.0.25:161 inside 10.0.0.30:1879, idle 0:00:27, bytes 706509, flags -
UDP outside 192.0.2.18:123 inside 10.0.0.108:123, idle 0:00:34, bytes 79, flags -
TCP outside 192.0.2.150:80 DMZ 172.16.0.5:59512, idle 0:00:00, bytes 0, flags Uf
TCP outside 192.0.2.150:80 inside 10.0.0.102:59559, idle 0:00:03, bytes 1488,
  flags UfRIO
...
```

Or to view more details:

```
ciscoasa# show conn detail
```

Another **state table** in ASA is the **local host table** which maintains per-host statistics as follows:

```
FIREWALL# show local-host 10.0.0.108
Interface management: 0 active, 0 maximum active, 0 denied
Interface DMZ: 2 active, 69 maximum active, 0 denied
Interface inside: 130 active, 320 maximum active, 0 denied
local host: <10.0.0.108>,
    TCP flow count/limit = 15/unlimited
    TCP embryonic count to host = 0
    TCP intercept watermark = unlimited
    UDP flow count/limit = 10/unlimited

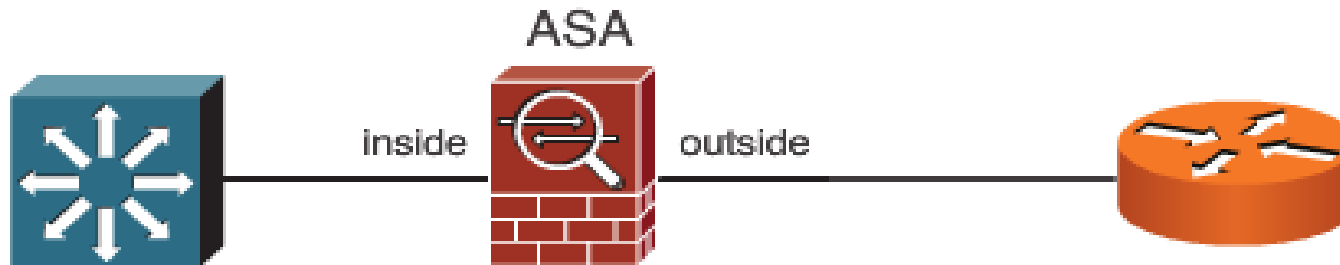
Xlate:
    NAT from any:10.0.0.108 to outside:209.165.200.238 flags I idle 0:00:00
    timeout 3:00:00

Conn:
    TCP outside 192.0.2.65:22 inside 10.0.0.108:50124, idle 0:00:58, bytes 443,
    flags UIO
    ... output omitted ...
Interface outside: 120 active, 1940 maximum active, 0 denied
```

Note that for connection with other network devices, ASAs will also need to be configured with **static** and/or **default routes**, or routing protocols to **learn routes dynamically**.

To configure **static route** in ASA:

```
ciscoasa(config)# route interface ip_address netmask gateway_ip
```



e.g.: Suppose your ISP can be reached via 23.1.2.7, you can configure a **static default route** in ASA as follows:

```
ciscoasa(config)# route outside 0.0.0.0 0.0.0.0 23.1.2.7
```

To verify the **routing table** in the **ASA**, use the similar command as in routers but note the **slight difference** as shown:

```
ciscoasa# show route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is 23.1.2.7 to network 0.0.0.0

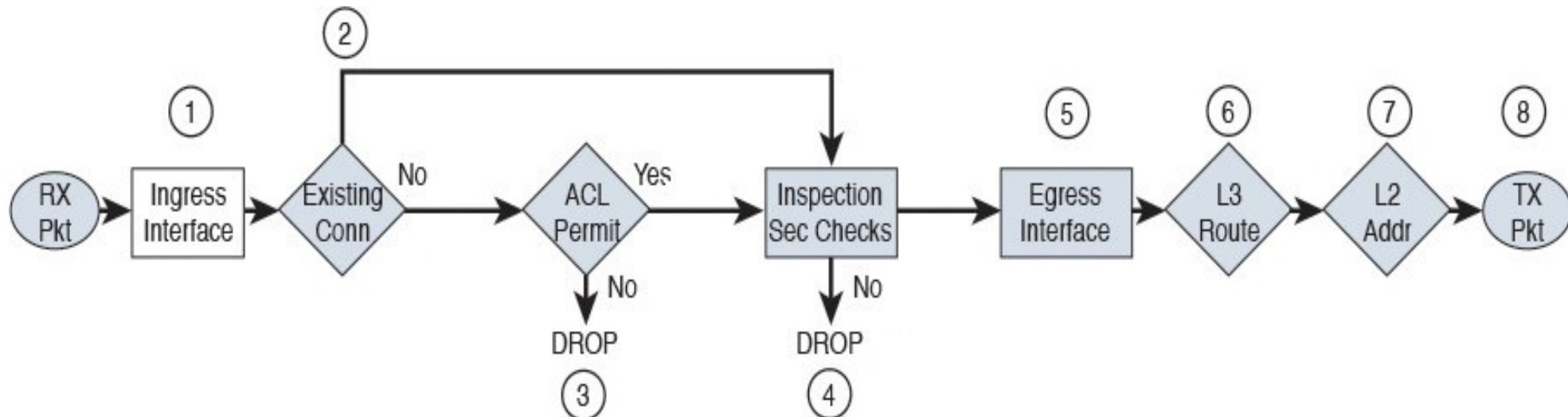
C    23.1.2.0 255.255.255.0 is directly connected, outside
C    192.168.1.0 255.255.255.0 is directly connected, management
S*   0.0.0.0 0.0.0.0 [1/0] via 23.1.2.7, outside
ciscoasa#
```

Before discussing the configuration of **access control policy** using **ACL** and **Modular Policy Framework (MPF)**, it's useful to understand packet flow in ASA.

When a packet arrives at ASA and before it is allowed to pass through, it will go through the following filtering as shown:

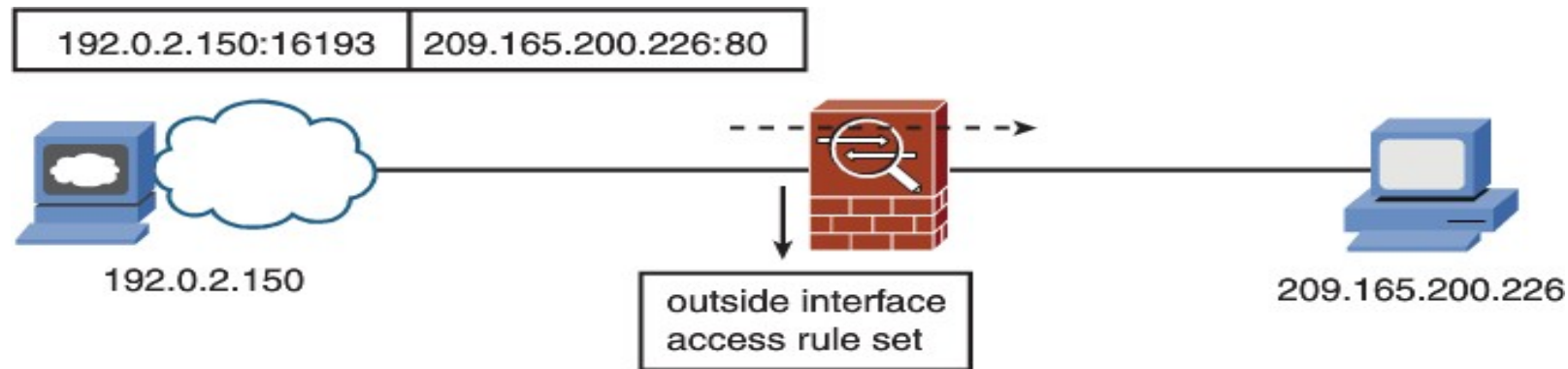
States table (maintain automatically after packets are permitted by **interface security level**)

Access control list (to override **interface security level** if needed) **MPF inspection security policy** (shipped with a factory default, but can be modified or replaced)



LAB 6 Part (2B) If needed, **ACLs** can be configured at ASA to override security level; e.g. to allow **lower security-level** to access **higher security-level**, or limit what higher-security level can send.

e.g.: To allow **outside** users to access web server 209.165.200.226 at DMZ



```
ciscoasa(config) # access-list OUTSIDE_TO_DMZ permit tcp any host 209.165.200.226 eq www
ciscoasa(config) # access-group OUTSIDE_TO_DMZ in interface outside
```

Note some **differences** between configuring **ACLs** in **ASAs** and routers:

1. ASAs assume ACLs are **extended** and 'extended' keyword is optional
2. ASAs use standard **subnet mask** but **not wildcard mask** to specify IP address block

Similar as Cisco IOS routers, **line numbers** are automatically added into **ACLs** in **ASAs** to support **editing** as follows:

To **show** ACL, use the command:

```
ciscoasa# show access-list OUTSIDE_TO_DMZ
access-list OUTSIDE_TO_DMZ;1elements;      name hash:0x5113f0a2
access-list OUTSIDE_TO_DMZ line 1 extended permit tcp any host
209.165.200.226 eq www (hitcnt=9) 0x4a02e414
```

To **insert** an ACL statement, use '**line**' keyword to indicate the position:

```
ciscoasa(config)# access-list OUTSIDE_TO_DMZ line 1 remark
allow internet access to web
```

To **delete** an ACL statement, use '**no**' keyword in front:

```
ciscoasa(config)# no access-list OUTSIDE_TO_DMZ line 1 remark
allow internet access to web
```

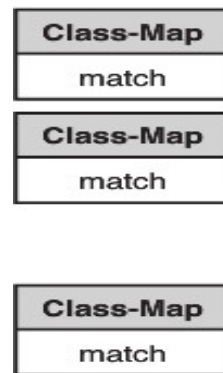
To **delete** the whole ACL:

```
ciscoasa(config)# clear configure access-list OUTSIDE_TO_DMZ
```

LAB 6 Part (2C) Access control policy in ASA can also be configured using Cisco Modular Policy Framework (MPF)

Configuring ASA using MPF involves 3 steps:

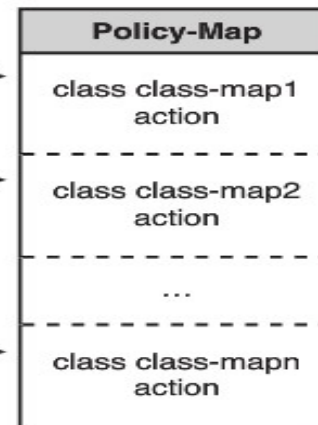
1. Configure **class-map**: specify traffic to be matched



Classify Traffic

Class Maps

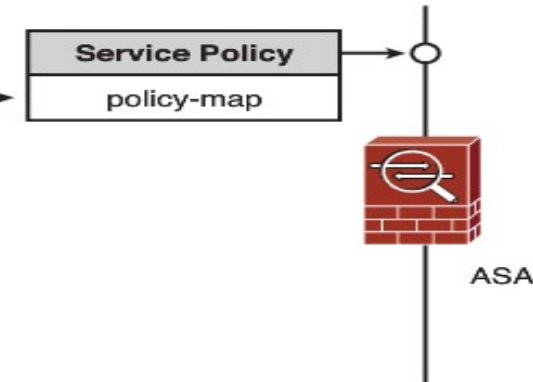
2. Configure **policy-map**: specify actions to be taken on **class-map** traffic



Define Actions

Policy Maps

3. Configure **service-policy**: apply **policy-map** to an interface or globally to all



Activate Policy

Service Policy

By default, ASA is pre-configured with a **class-map** named **inspection-default** and a **policy-map** named **global_policy** which is applied globally using **service-policy** on all ASA interfaces.

<output omitted>

```
class-map inspection_default  
  match default-inspection-traffic
```

Class map consists of one statement matching a special keyword **default-inspection-traffic**.

```
policy-map global_policy  
  class inspection_default  
    inspect dns preset_dns_map  
    inspect ftp  
    inspect h323 h225  
    inspect h323 ras  
    inspect ip-options  
    inspect netbios  
    inspect rsh  
    inspect rtsp  
    inspect skinny  
    inspect esmtp  
    inspect sqlnet  
    inspect sunrpc  
    inspect tftp  
    inspect sip  
    inspect xdmcp
```

Policy map associates actions to perform on the traffic identified in the class map.

```
service-policy global_policy global
```

Service policy applies a policy map to an interface or to all interfaces using the keyword **global**. The **global** keyword applies a policy map to interfaces that do not have a specific policy applied.

<output omitted>

Specifically, the **default-inspection-traffic** in **class-map inspection_default** is matching commonly encountered network traffic as follows:

default-inspection-traffic Match default inspection traffic:

ctiqbe----tcp--2748	dns-----udp--53
ftp-----tcp--21	gtp-----udp--2123,3386
h323-h225-tcp--1720	h323-ras--udp--1718-1719
http-----tcp--80	icmp-----icmp
ils-----tcp--389	ip-options-----rsvp
mgcp-----udp--2427,2727	netbios---udp--137-138
radius-acct---udp--1646	rpc-----udp--111
rsh-----tcp--514	rtsp-----tcp--554
sip-----tcp--5060	sip-----udp--5060
skinny---tcp--2000	smtp-----tcp--25
sqlnet---tcp--1521	tftp-----udp--69
waas-----tcp--1-65535	xmcp-----udp--177

To view **statistics** of the matching traffic inspected by the **ASA**, use the show command as follows:



```
ciscoasa# show service-policy
Global policy:
  Service-policy: global_policy
    Class-map: inspection_default
      Inspect: dns preset_dns_map, packet 0, drop 0, reset-drop 0
      Inspect: ftp, packet 0, drop 0, reset-drop 0
      Inspect: h323 h225 _default_h323_map, packet 0, drop 0, reset-drop 0
             tcp-proxy: bytes in buffer 0, bytes dropped 0
      Inspect: h323 ras _default_h323_map, packet 0, drop 0, reset-drop 0
      Inspect: rsh, packet 0, drop 0, reset-drop 0
      Inspect: rtsp, packet 0, drop 0, reset-drop 0
             tcp-proxy: bytes in buffer 0, bytes dropped 0
      Inspect: esmtp _default_esmtp_map, packet 0, drop 0, reset-drop 0
      Inspect: sqlnet, packet 0, drop 0, reset-drop 0
      Inspect: skinny, packet 0, drop 0, reset-drop 0
             tcp-proxy: bytes in buffer 0, bytes dropped 0
      Inspect: sunrpc, packet 0, drop 0, reset-drop 0
             tcp-proxy: bytes in buffer 0, bytes dropped 0
      Inspect: xdmcp, packet 0, drop 0, reset-drop 0
      Inspect: sip, packet 0, drop 0, reset-drop 0
             tcp-proxy: bytes in buffer 0, bytes dropped 0
      Inspect: netbios, packet 0, drop 0, reset-drop 0
      Inspect: tftp, packet 0, drop 0, reset-drop 0
      Inspect: ip-options _default_ip_options_map, packet 0, drop 0, reset-drop 0
```

To view **statistics** of the matching traffic inspected by the **ASA**, use the show command as follows:

E.g.: For security reason, ICMP traffic is denied in ASA by default. But if your company's **security policy** allows ICMP, you can configure ASA to maintain stateful tracking of ICMP to permit return ICMP traffic by adding **ICMP inspection engine** in the default policy-map as shown below:

```
ciscoasa(config)# policy-map global_policy
ciscoasa(config-pmap)# class inspection_default
ciscoasa(config-pmap-c)# inspect icmp
ciscoasa(config-pmap-c)# exit
ciscoasa(config-pmap)# exit
ciscoasa(config)#
```

Note: You may also write your own policy-map and apply it on appropriate interface of the ASA.

Alternatively, to configure new MPF:

Step 1: Configure MPF class map to specify what type of network traffic to be matched according to some conditions

For Example,

```
ciscoasa(config)# class-map class_map_name
ciscoasa(config-cmap)# description text
ciscoasa(config-cmap)# match ...
ciscoasa(config-cmap)# exit
```

Common match commands for specifying traffic:

Matching Condition	Command Syntax
Any traffic	ciscoasa(config-cmap)# match any
Default traffic types	ciscoasa(config-cmap)# match default-inspection-traffic
Destination port number	ciscoasa(config-cmap)# match port {tcp udp} {eq port range start end}
Access list	ciscoasa(config-cmap)# match access-list acl_name
RTP port number range	ciscoasa(config-cmap)# match rtp starting_port range

An example of configuring **MPF class map**:

```
ciscoasa(config)# class-map anything
ciscoasa(config-cmap)# match any
ciscoasa(config-cmap)# exit
!
ciscoasa(config)# class-map voice
ciscoasa(config-cmap)# match rtp 2000 100
ciscoasa(config-cmap)# exit
!
ciscoasa(config)# access-list dc extended permit ip any 10.100.0.0 255.255.0.0
ciscoasa(config)# class-map data-center
ciscoasa(config-cmap)# match access-list dc
ciscoasa(config-cmap)# exit
```

Step 2: Configure MPF layer 3/4 policy map to specify one or more actions to take on the matched traffic

For Example,

```
ciscoasa(config)# policy-map policy_map_name
ciscoasa(config-pmap)# description text
ciscoasa(config-pmap)# class class_map_name
ciscoasa(config-pmap-c)# ...
```

Some commands for configuring **actions** to take on matched traffic:

Action	Command Syntax
Set connection limits	ciscoasa(config-pmap-c)# set connection ...
Inspect applications	ciscoasa(config-pmap-c)# inspect <i>engine_name</i>
Police the traffic	ciscoasa(config-pmap-c)# police {output input} <i>conform_rate</i> [<i>burst_bytes</i>] [conform-action {drop transmit}] [exceed-action {drop transmit}]
Apply priority handling	ciscoasa(config-pmap-c)# priority

An example of configuring **MPF layer 3/4 policy map** based on previously configured class maps:

```
ciscoasa(config)# policy-map p1
ciscoasa(config-pmap)# class anything
ciscoasa(config-pmap-c)# set connection ...
ciscoasa(config-pmap-c)# inspect ...
ciscoasa(config-pmap-c)# exit
ciscoasa(config-pmap)# class voice
ciscoasa(config-pmap-c)# priority
ciscoasa(config-pmap-c)# exit
ciscoasa(config-pmap)# class data-center
ciscoasa(config-pmap-c)# set connection timeout ...
ciscoasa(config-pmap)# exit
```

Note: When multiple classes are matched:

- If actions are **different**, **apply all**; e.g. traffic matching 'data-center' will also match 'anything', then actions 'set connection ...', 'inspect ...' and 'set connection timeout ...' will be applied.
- If actions are the **same**, **apply the first**.

Step 3: Configure **MPF service policy** to apply policy map to the ASA interface(s) as follows:

```
ciscoasa(config)# service-policy policy_map_name {global | interface if_name}
```

For example,

```
ciscoasa(config)# service-policy p1 interface outside
```

Service policy may be applied **individually** to a particular interface, or **globally** to all interfaces. Also note the direction of traffic applied to:

- Each interface can have **only one service-policy** applied to it
- In addition, **only one global policy** can be applied to all interfaces
- **Interface service-policy** takes precedence over **global service-policy**

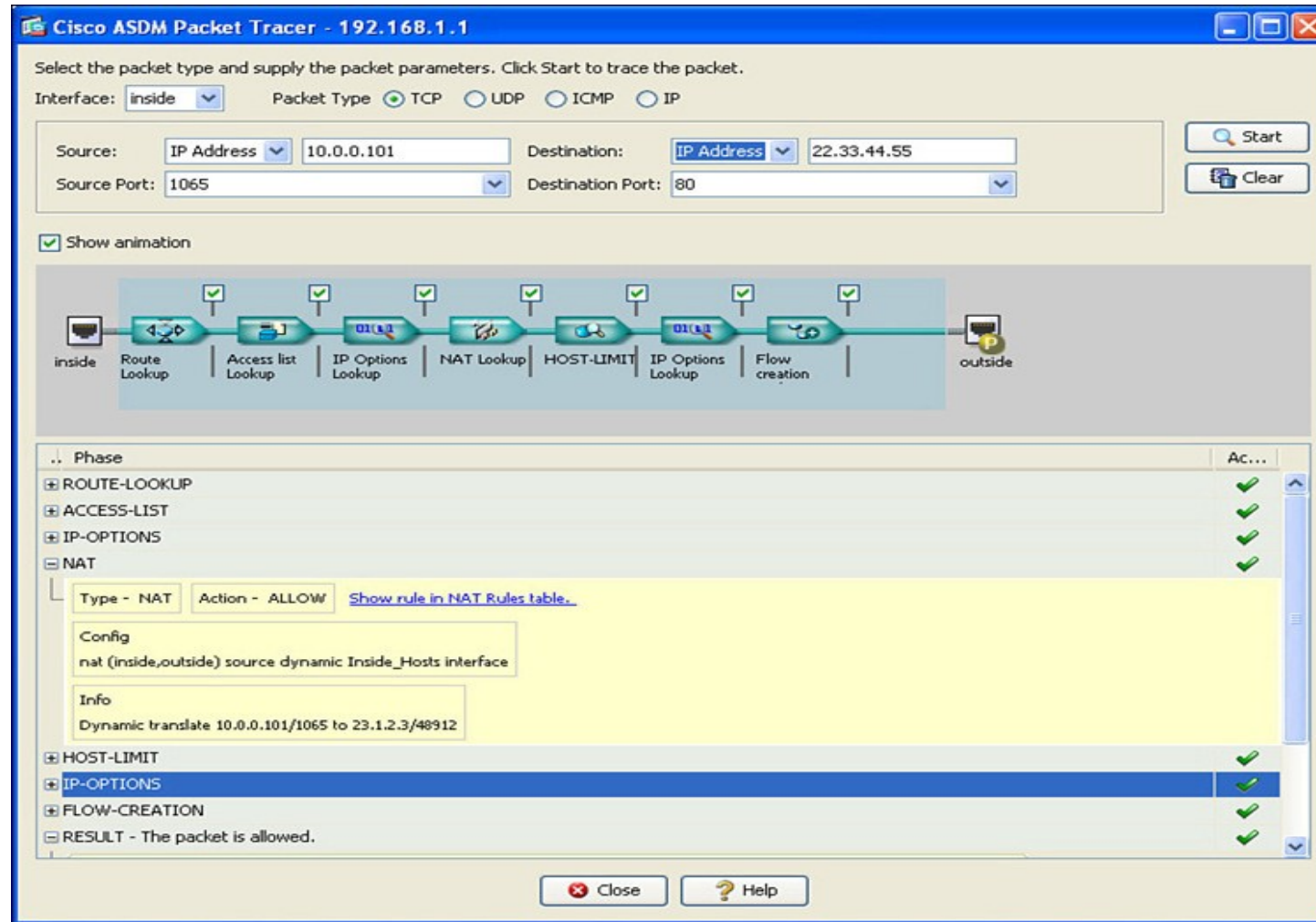
Action	Applied to Interface	Applied Globally
Set connection limits	Bidirectional	Ingress only
Inspect with application engines	Bidirectional	Ingress only
Policing (input)	Ingress only	Ingress only
Policing (output)	Egress only	Egress only
Priority handling	Egress only	Egress only

LAB 6 Part (2D) (Optional) Cisco has developed a GUI called ASDM (Adaptive Security Device Manager) for configuring ASA which you may try in the lab



ASDM can be accessed via web browser to HTTPS server running in ASA through the **Management 0/0 interface** with a factory default IP address 192.168.1.1/24.

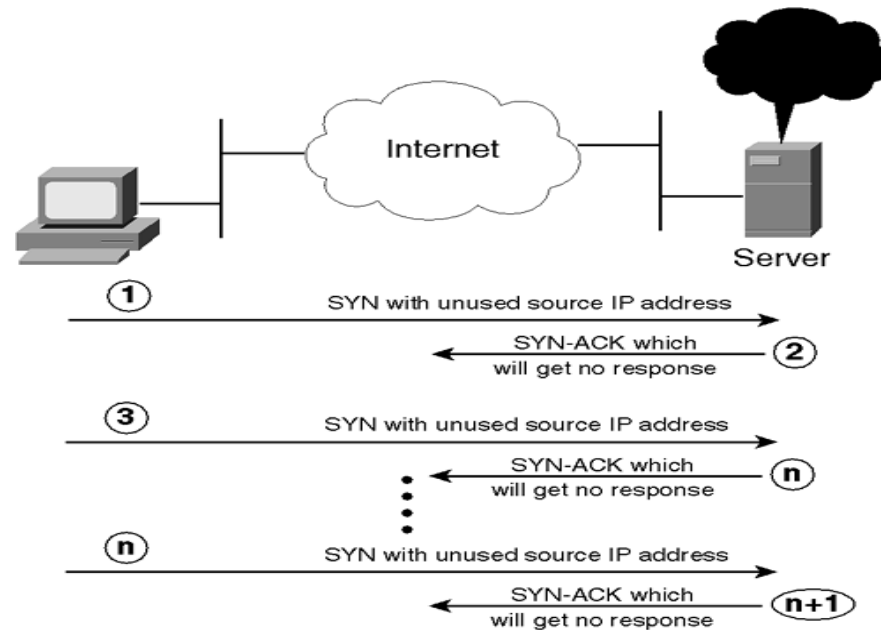
ASDM has a nice feature called **packet tracer** (note **different** from **Cisco Academy** packet tracer) that allows you to trace whether packets will be dropped based on your configurations



LAB 6 Part 3

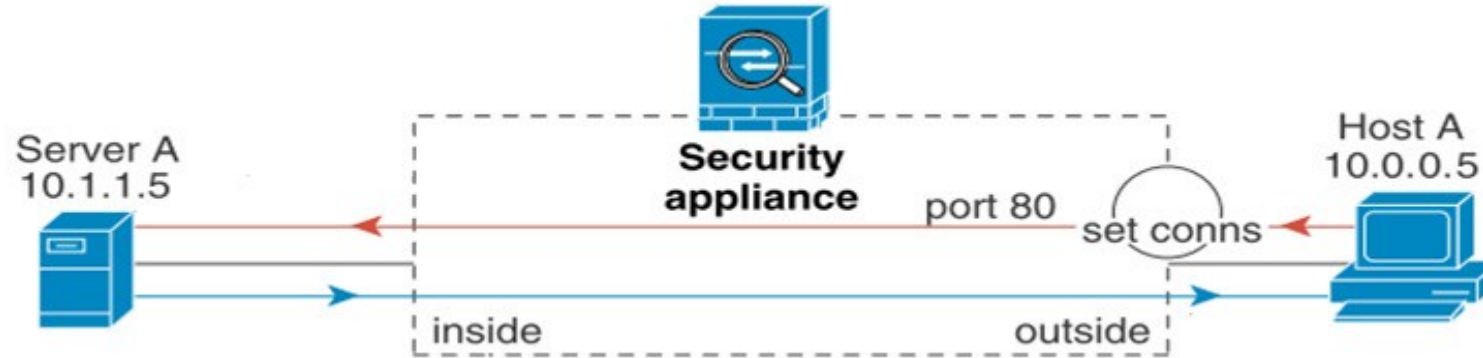
- Configure **MPF inspection security policy** in ASA to defend against **TCP SYN attack**

TCP SYN attack is based on **not completing 3-way TCP handshake**, thus leaving server with many connections waiting to be established called **embryonic connections**.



```
# hping3 -c 15000 -d 120 -S -w 64 -p 80 --flood --rand-source <server IP>
```

To defend **TCP SYN attack**, we can use **MPF** to **limit** the maximum number of **embryonic connections** which when **exceeded** will trigger **TCP intercept** automatically in ASA.



Step 1: Create **class map** to specify server to protect

```
ciscoasa(config)# access-list serverA extended permit tcp any host 10.1.1.5 eq 80
ciscoasa(config)# class-map http_serverA
ciscoasa(config-cmap)# match access-list serverA
```

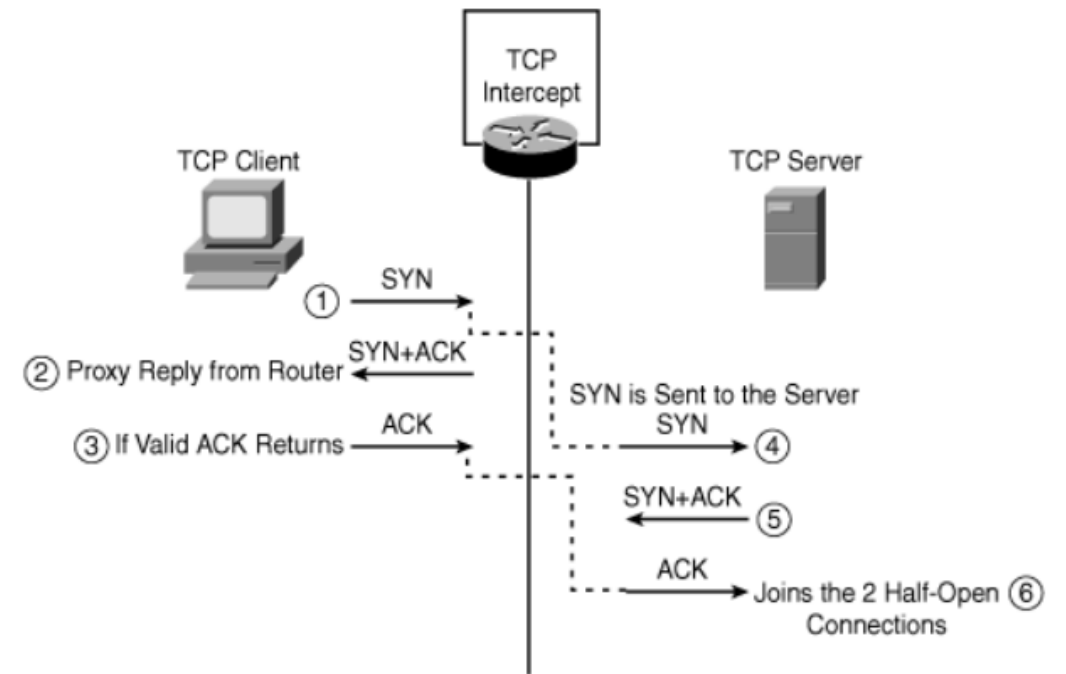
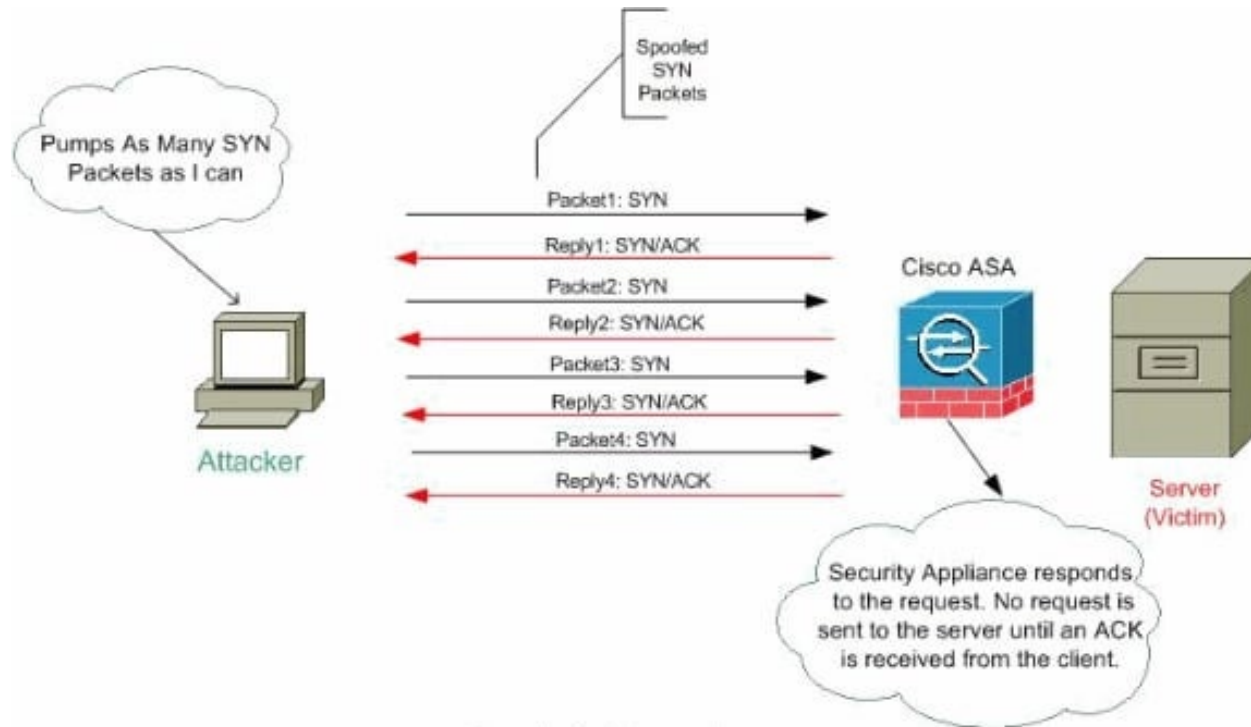
Step 2: Create **policy map** to specify actions to take

```
ciscoasa(config)# policy-map policy_serverA
ciscoasa(config-pmap)# class http_serverA
ciscoasa(config-pmap-c)# set connection embryonic-conn-max 100
ciscoasa(config-pmap-c)# set connection per-client-embryonic-max 10
```

Step 3: Apply **policy map** on appropriate interface

```
ciscoasa(config)# service-policy policy_serverA interface outside
```

Cisco ASA: TCP intercept



To verify and monitor attacks intercepted by **TCP intercept**, you may configure **threat detection statistics** as follows:

- Enabling **threat detection statistics** for TCP intercept:

```
ciscoasa(config)# threat-detection statistics tcp-intercept
rate-interval 3 burst-rate 60 average-rate 30
```

- **rate-interval** sets the size of monitoring window in **mins** during which ASA samples the number of attacks 60 times
- **burst-rate** and **average-rate** are in attacks/events per sec (**eps**)
 - sets the threshold for syslog message generation

- Monitoring **TCP intercept statistics**:

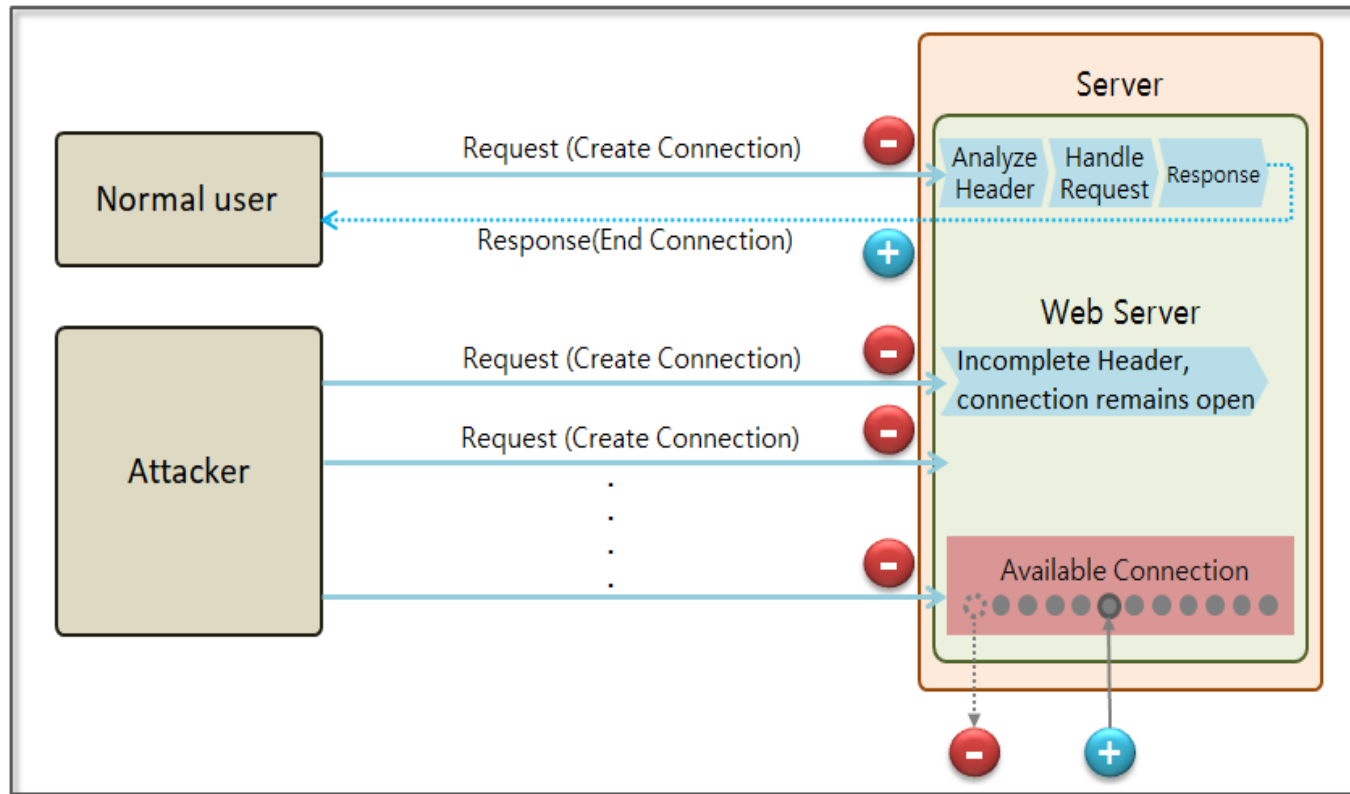
```
ciscoasa(config)# show threat-detection statistics top tcp-intercept
Top 10 protected servers under attack (sorted by average rate)
Monitoring window size: 3 mins      Sampling interval: 3 secs
<Rank> <Server IP:Port> <Interface> <Ave Rate> <Cur Rate> <Total> <Source IP (Last Attack Time)>
-----
1    10.1.1.5:80 inside 1249 9503 2249245 10.0.0.3 (0 secs ago)
```

LAB 6 Part 4

- ➡ Configure **MPF inspection security policy** in ASA to defend against **Slowloris attack**

Slowloris attack is an example of **DoS** attack on web servers using low-rate but maliciously crafted packets to exploit design weakness of HTTP protocol

By design, web server will only process HTTP request when header is received completely.



Slowloris attack is generated by creating:

1. **Multiple concurrent connections** to the web server, &
2. For each connection, intentionally **sending HTTP header slowly**
3. In multiple **small fragments**,

Slowloris is thus able to consume all connections and make the server unavailable.

To try **Slowloris attack** in the lab, you may use the **slowhttptest** in Kali Linux.

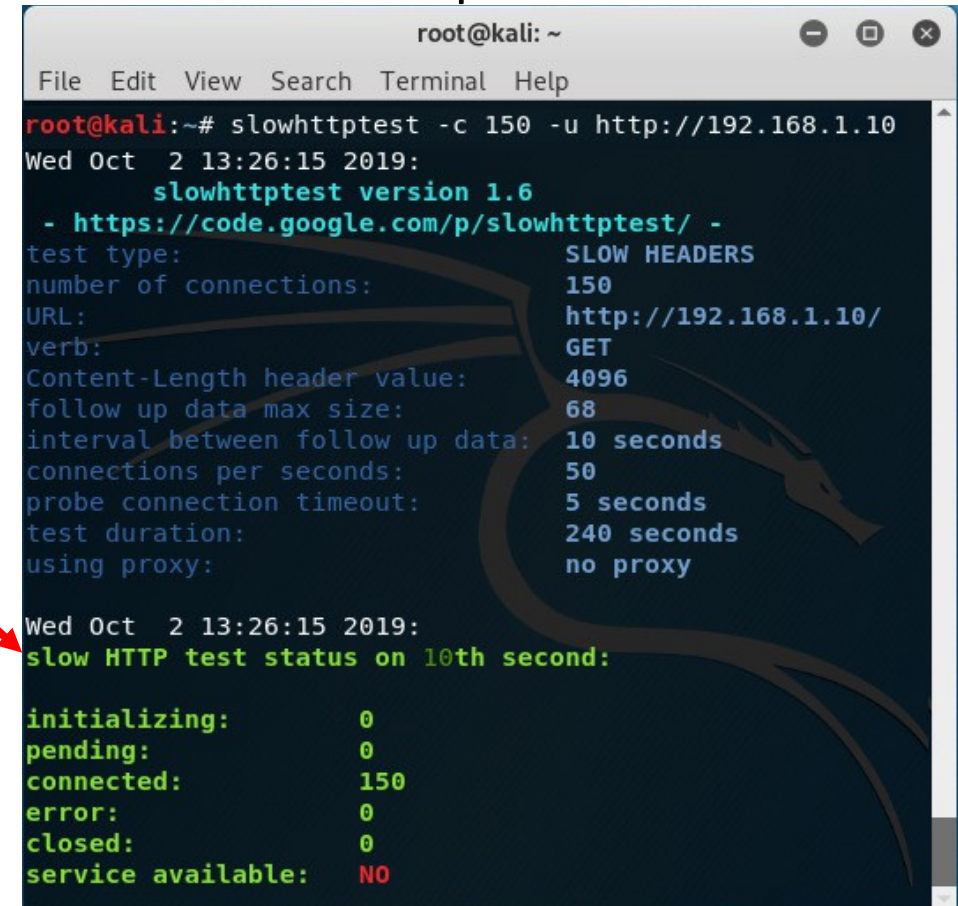
```
root@kali:~# slowhttptest -c number_connections -u target_url
```

Refer <https://tools.kali.org/stress-testing/slowhttptest> on the use of slowhttptest.

An example of **Slowloris attack** on XAMPP Apache web server with default setting.

Notice that a single client making 150 connections is sufficient to make Apache web server unavailable!

A possible defense is to configure **MPF inspection security policy** similar to defending TCP SYN attack. Try it!



```
root@kali: ~
File Edit View Search Terminal Help

root@kali:~# slowhttptest -c 150 -u http://192.168.1.10
Wed Oct 2 13:26:15 2019:
    slowhttptest version 1.6
    - https://code.google.com/p/slowhttptest/ -
test type:                                SLOW HEADERS
number of connections:                    150
URL:                                      http://192.168.1.10/
verb:                                     GET
Content-Length header value:              4096
follow up data max size:                  68
interval between follow up data:         10 seconds
connections per seconds:                 50
probe connection timeout:                5 seconds
test duration:                          240 seconds
using proxy:                             no proxy

Wed Oct 2 13:26:15 2019:
slow HTTP test status on 10th second:

initializing:      0
pending:           0
connected:         150
error:             0
closed:            0
service available: NO
```